



# 第 5 章

## Trees

## 树

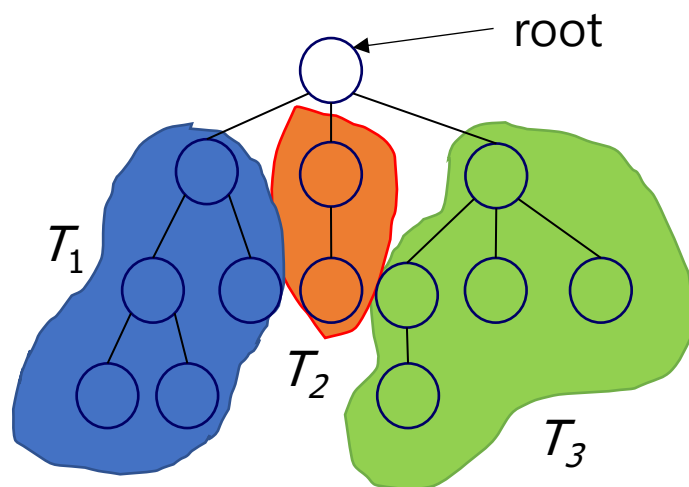
# 内容

- 简介
- 二叉树
- 二叉树遍历
- 二叉树其他操作
- 线索二叉树
- 堆
- 二叉搜索树

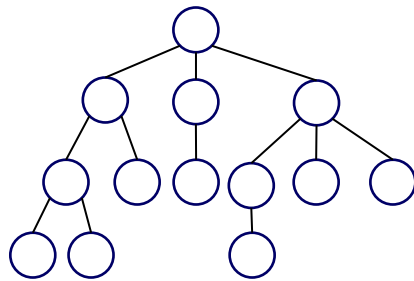
# 树-Trees

❖ 定义: 树是由一个或多个节点组成的有限集合:

- 有一个特殊节点叫做根节点(root)
- 其余节点被分为  $n$  ( $n \geq 0$ ) 互不相交的集合  $T_1, \dots, T_n$ , 每个集合本身是一棵树。  $T_1, \dots, T_n$  被称为根的子树。



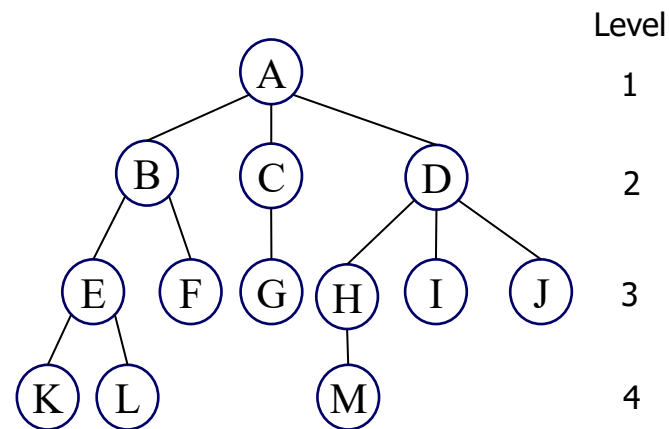
# 术语



- ❖ **节点**：包含信息项以及指向其他节点的枝干。
- ❖ **节点的度**：一个节点所拥有的子树数量。
- ❖ **树的度**：树中所有节点度的最大值。
- ❖ **终端节点（或叶节点）**：度为零的节点。
- ❖ **父节点与子节点**：拥有子树的节点是其子树根节点的父节点，而子树的根节点是该节点的子节点。
- ❖ **兄弟节点**：拥有相同父节点的子节点。
- ❖ **节点的祖先**：从树根到该节点路径上的所有节点。
- ❖ **节点的后代**：该节点所有子树中的节点。
- ❖ **节点的层级**：其父节点的层级加一（根节点位于第 1 层）。
- ❖ **树的高度（或深度）**：树中节点的最大层级。

# 示例

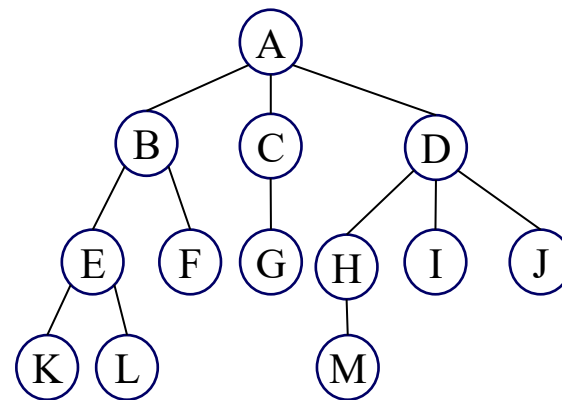
- ❖ 树的根节点: A
- ❖ 节点A的度: 3
- ❖ 终端节点(或叶子节点): {K, L, F, G, M, I, J}
- ❖ 节点D的父节点: A
- ❖ 节点D的子节点: H, I, J
- ❖ 节点H的兄弟节点: I, J
- ❖ 树的高度(或深度): 4
- ❖ 树的度: 3
- ❖ 节点M的祖先节点: A, D, H
- ❖ 节点D的后代节点: H, I, J, M



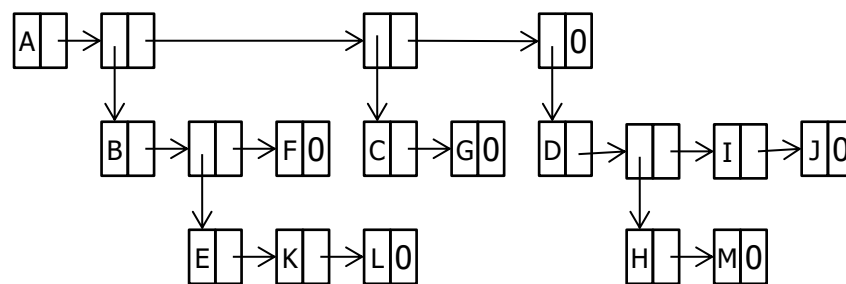
# 树的表示

## ❖ 列表List(表示)

- 将树写成一个列表，其中每个子树也是一个列表
- 首先是根节点中的信息，然后是该节点的子树列表



(A(B(E(K,L),F),C(G),D(H(M),I,J)))



树的列表表示

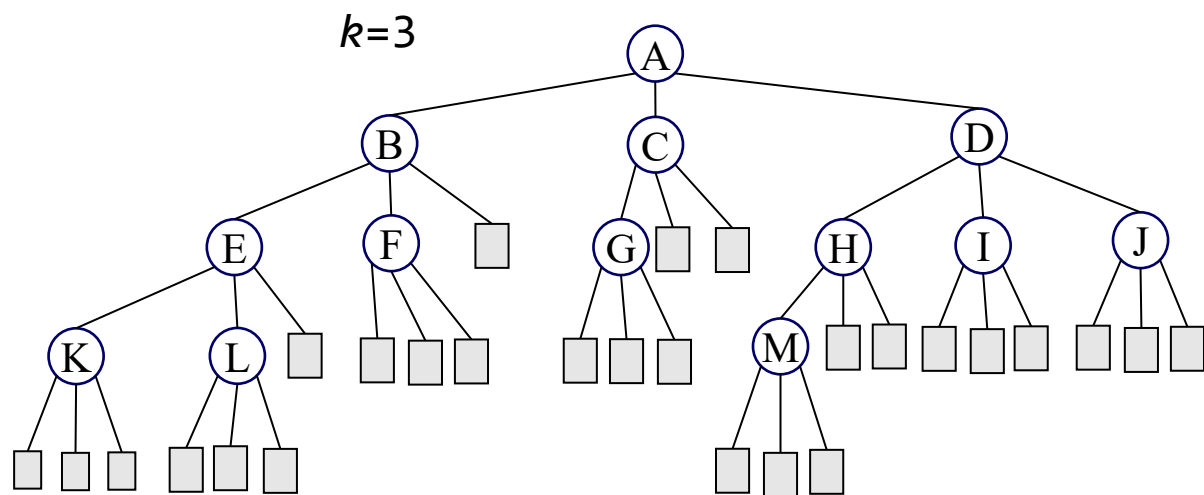
# 树的表示

## ❖ k叉树的表示法

| DATA | CHILD1 | ... | CHILD $k$ |
|------|--------|-----|-----------|
|------|--------|-----|-----------|

k度树的可能节点结构

如果  $T$  是一棵  $k$  叉树，包含  $n$  个节点 ( $n \geq 1$ )，且每个节点的大小固定。则在  $n \cdot k$  个子节点字段中，有  $n \cdot (k - 1) + 1$  个字段为  $\emptyset$  (表示空子节点)。



# 非空子节点字段的数量 =  $n - 1$

# 子节点字段总数 =  $n \cdot k$

# 空子节点字段的数量

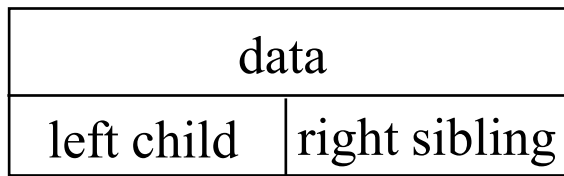
$$= n \cdot k - (n - 1)$$

$$= n \cdot (k - 1) + 1$$

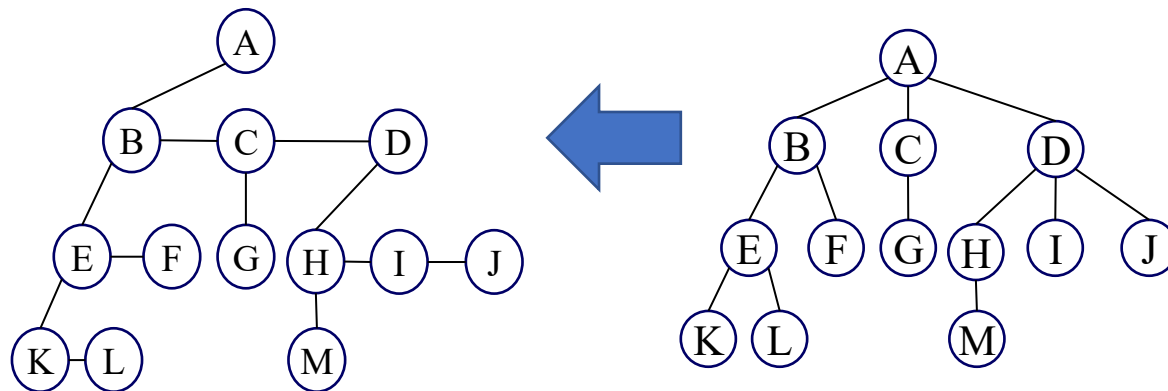
# 树的表示

## ❖ 左子右兄表示法

- 这种表示法使处理固定大小的节点更简单
- 每个节点最多有一个最左子节点和一个最近的右兄弟节点
  - 节点A最左边的子节点: B
  - 节点B最近的右兄弟节点: C



左子右兄节点结构

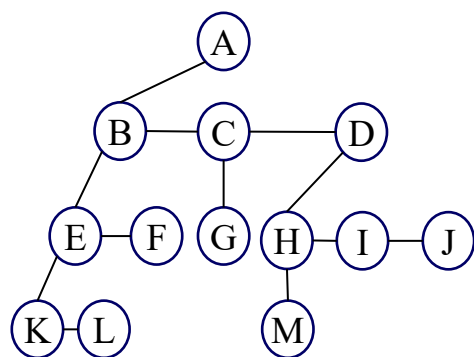


- 由于树中子节点的顺序不重要，任何子节点都可以作为最左子节点，其任意兄弟节点可以作为最近的右兄弟节点。



# 表示度为2的树

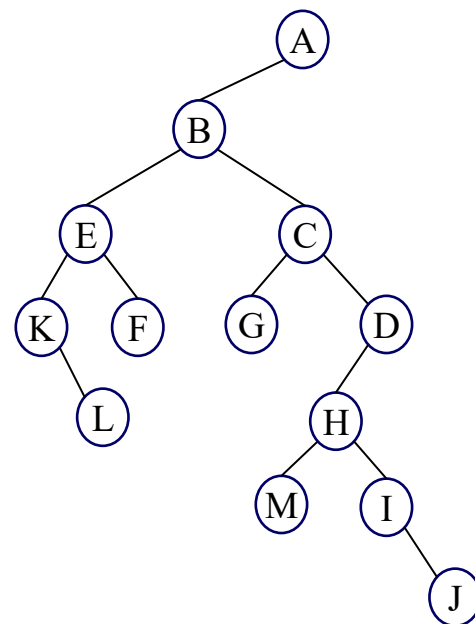
❖ 将左子右兄树中的右兄指针顺时针旋转 45 度



左子右兄树



顺时针旋转45度



左子右子树 = 二叉树

# 二叉树

## ❖ 定义 (使用递归)

➤ 二叉树是一个有限的节点集合，可以为空，也可以由一个根节点和两个互不相交的二叉树组成，这两个二叉树分别称为左子树和右子树。

❖ 二叉树的关键要求是任意节点的度不得超过2

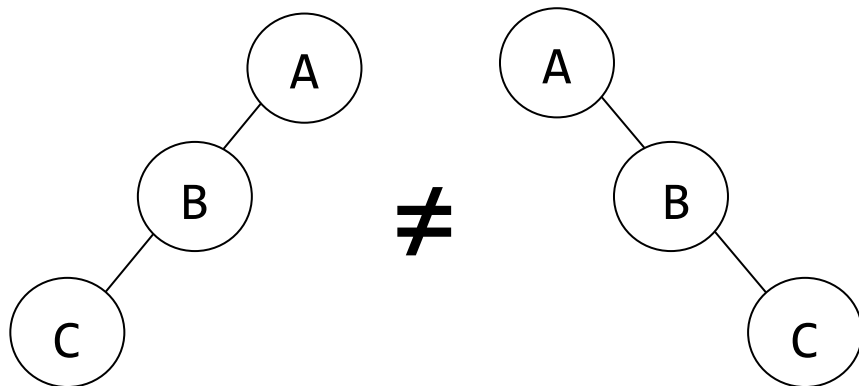
❖ 左子树和右子树是有明确区分的

❖ 任何树都可以通过左子右兄表示法转换为二叉树

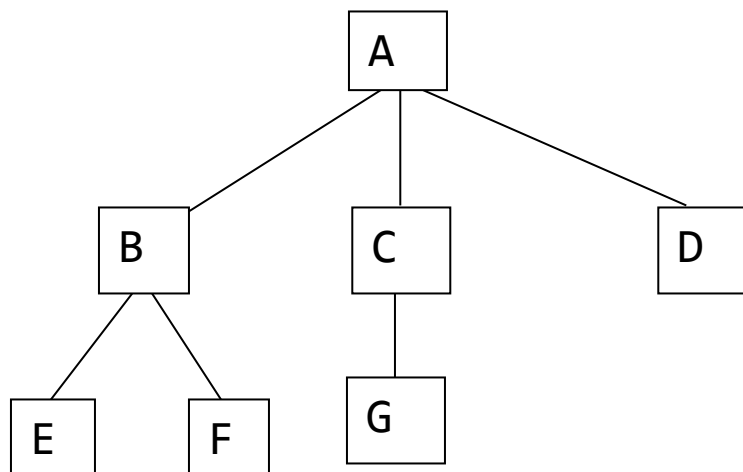
# 二叉树

## ❖ 二叉树与树的不同之处

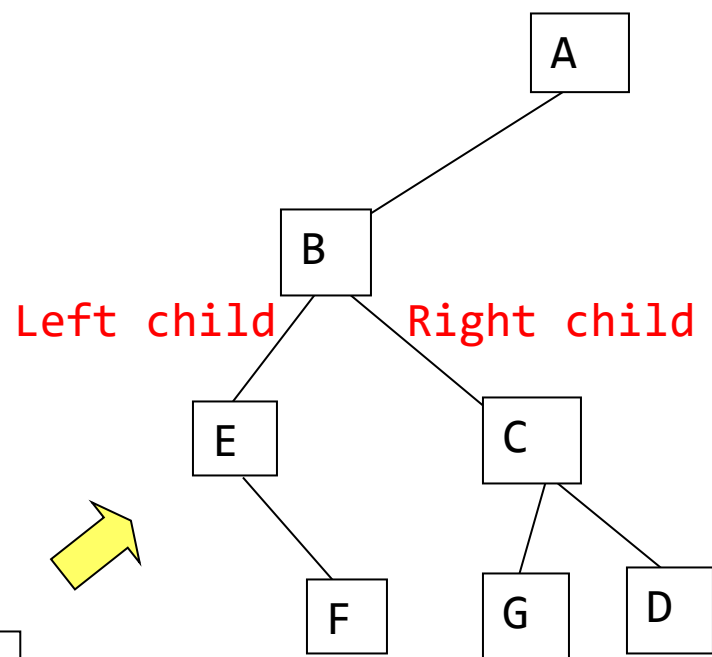
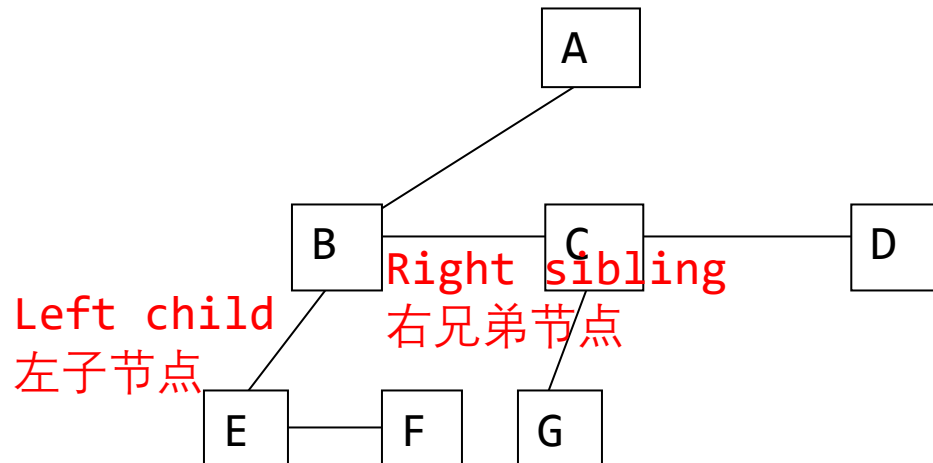
- 子节点数量限制：树无限制；二叉树最多2个
- 子节点顺序的重要性：树可能无序(取决于定义)，二叉树有序(左右子树有明确区分)



# 二叉树



左子右兄表示



# 抽象数据类型：二叉树

**ADT Binary\_Tree(abbreviated BinTree) is**

**objects :** 一个有限的节点集合，可以为空，也可以由一个根节点、左二叉树和右二叉树组成

**functions :** for all  $bt, bt1, bt2 \in \text{BinTree}$ ,  $item \in \text{element}$

*BinTree* Create()                   ::= 创建一个空二叉树

*Boolean* IsEmpty(*bt*)           ::= **if** (*bt*==空二叉树) **return** *TRUE*  
                                      **else return** *FALSE*

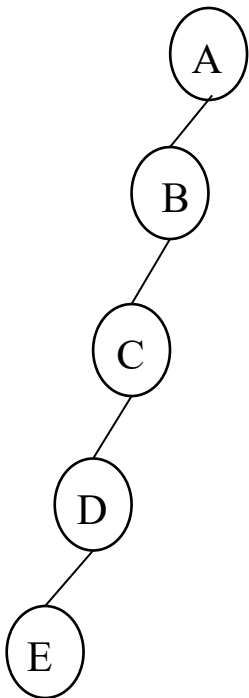
*BinTree* MakeBT(*bt1, item, bt2*)::= **return** 一个(左子树为*bt1*右子树为*bt2*,并且根节点包含数据*item*的)二叉树

*BinTree* Lchild(*bt*)               ::= **if** (IsEmpty(*bt*)) **return** error  
                                      **else return** *bt* 的左子树

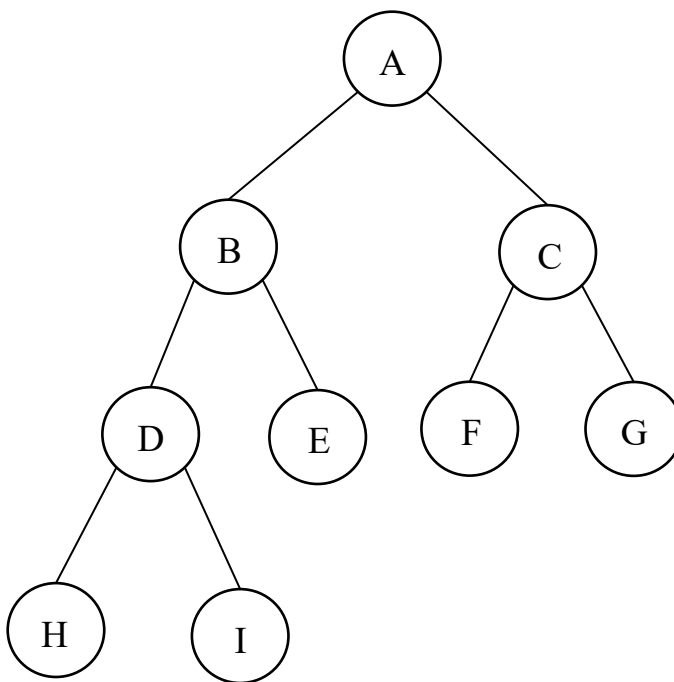
*element* Data(*bt*)                ::= **if** (IsEmpty(*bt*)) **return** error  
                                      **else return** 二叉树*bt*根节点中的数据

*BinTree* Rchild(*bt*)               ::= **if** (IsEmpty(*bt*)) **return** error  
                                      **else return** *bt* 的右子树

# 两种特殊类型的二叉树



斜二叉树



完全二叉树

# 二叉树的性质

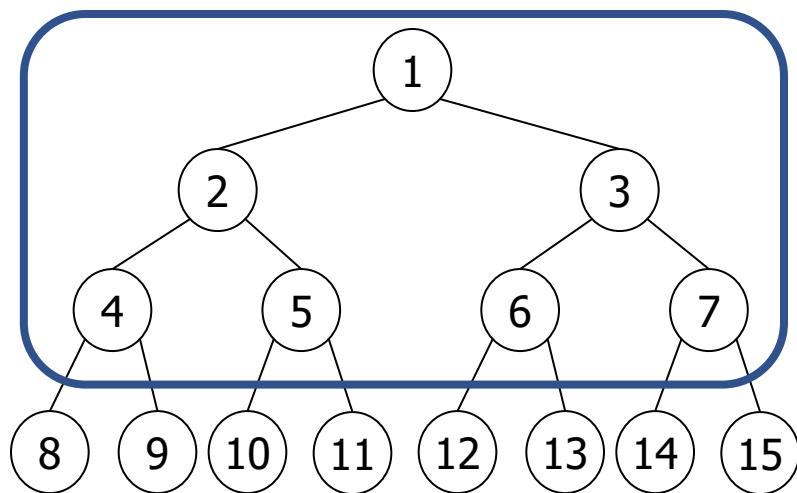
## ❖ 二叉树中的最大节点数

- 性质1: 二叉树第  $i$  层的最大节点数为  $2^{i-1}$ ,  $i \geq 1$
- 性质2: 深度为  $k$  的二叉树的最大节点数为  $2^k - 1$ ,  $k \geq 1$
- 证明 (归纳法)
  - 基础
    - 根节点是第  $i=1$  层的唯一节点。因此, 第  $i=1$  层的最大节点数为  $2^{i-1} = 2^0 = 1$
  - 假设
    - 设  $i$  为任意大于 1 的正整数. 假设第  $i-1$  层的最大节点数为  $2^{i-2}$
  - 步骤
    - 根据归纳假设, 第  $i-1$  层的最大节点数为  $2^{i-2}$ 。由于二叉树中每个节点的最大度为 2, 第  $i$  层的最大节点数是第  $i-1$  层最大节点数的两倍, 即  $2 * 2^{i-2} = 2^{i-1}$

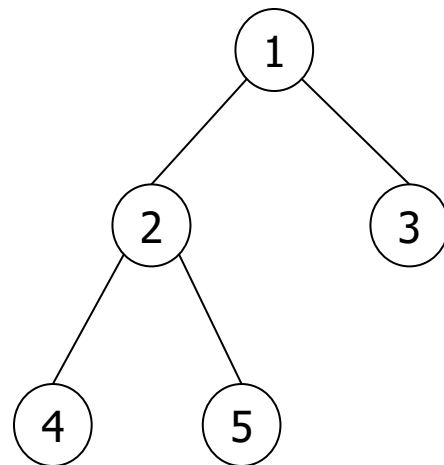
$$\sum_{i=1}^k (\text{第 } i \text{ 层的最大节点数}) = \sum_{i=1}^k 2^{i-1} = 2^k - 1$$

# 满二叉树与完全二叉树

- ❖ 定义: 深度为 $k$ 的**满二叉树**是一棵深度为 $k$ 且拥有  $2^k - 1$  个节点的二叉树, 其中 $k \geq 0$
- ❖ 定义: 一棵有 $n$ 个节点且深度为 $k$ 的二叉树, 当且仅当其节点对应于深度为 $k$ 的满二叉树中编号从1到 $n$ 的节点时, 才是**完全二叉树**。



深度为4的满二叉树，带有顺序  
节点编号



完全二叉树

一个有  $n$  个节点的完全二叉树的高度为  $\log_2(n + 1)$ ,  
其中  $\lceil x \rceil$  表示大于等于  $x$  的最小整数。

由前述二叉树性质,可知:

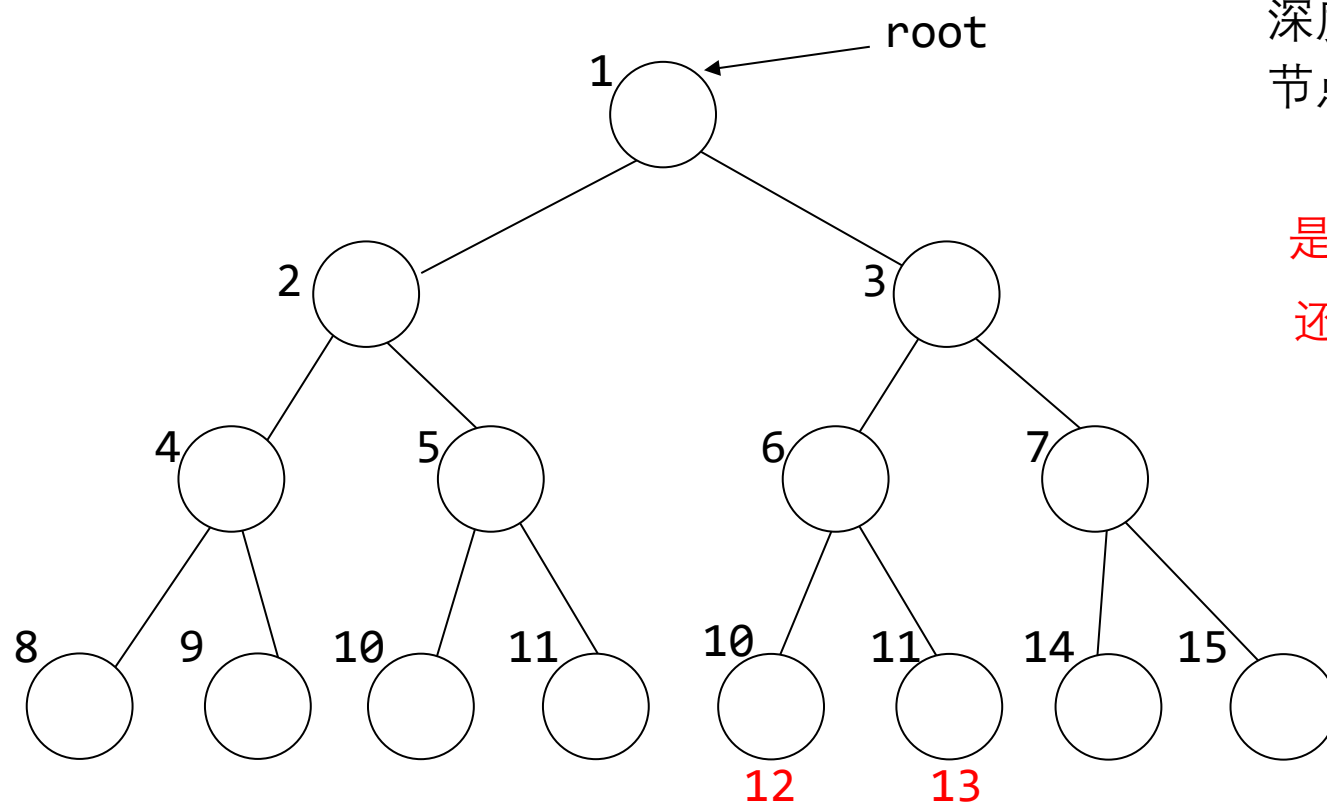
$$n = 2^k - 1$$

$$n + 1 = 2^k$$

$$\log_2(n + 1) = k$$



# 示例



深度 = 4

节点总数 =  $15 = 2^4 - 1$

是满二叉树?

还是完全二叉树 ?

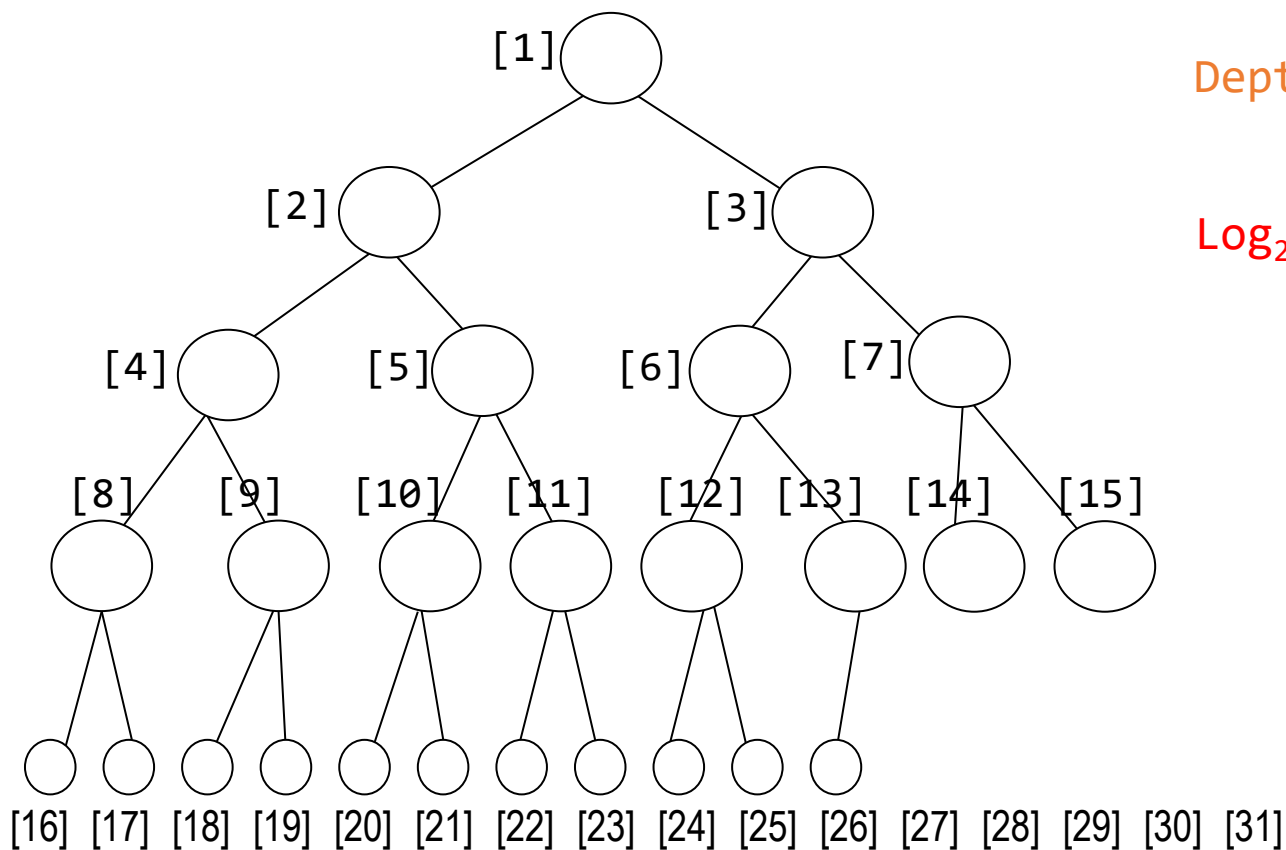
# 数组表示法

❖ 如果一个有  $n$  个节点的完全二叉树采用顺序表示, 则对于索引为  $i$  ( $1 \leq i \leq n$ ), 有如下规则:

- 父节点  $\text{parent}(i)$ : 如果  $i \neq 1$ , 则  $\text{parent}(i)$  位于  $\lfloor i/2 \rfloor$ ; 如果  $i = 1$ ,  $i$  是根节点, 没有父节点。
- 左子节点  $\text{leftChild}(i)$ : 如果  $2i \leq n$ , 则  $\text{leftChild}(i)$  位于  $2i$ . 如果  $2i > n$ , 则  $i$  没有左子节点。
- 右子节点  $\text{rightChild}(i)$ : 如果  $2i+1 \leq n$ , 则  $\text{rightChild}(i)$  位于  $2i+1$ ; 如果  $2i+1 > n$ , 则  $i$  没有右子节点。

# 示例

parent( $i$ ) is at  $\lfloor i/2 \rfloor$  if  $i \neq 1$ . 如果  $i=1$ ,  $i$  是根节点, 没有父节点。  
leftChild( $i$ ) is at  $2i$  if  $2i \leq n$ . 如果  $2i > n$ , 则  $i$  没有左子节点。  
rightChild( $i$ ) is at  $2i+1$  if  $2i+1 \leq n$ . 如果  $2i+1 > n$ , 则  $i$  没有右子节点。



$$n = 26$$

$$\text{Depth(深度)} = 5$$

$$= \lceil \log_2(26 + 1) \rceil$$

$$\log_2 2^{5-1} < \log_2 27 < \log_2 2^5$$

5 的父节点 ?

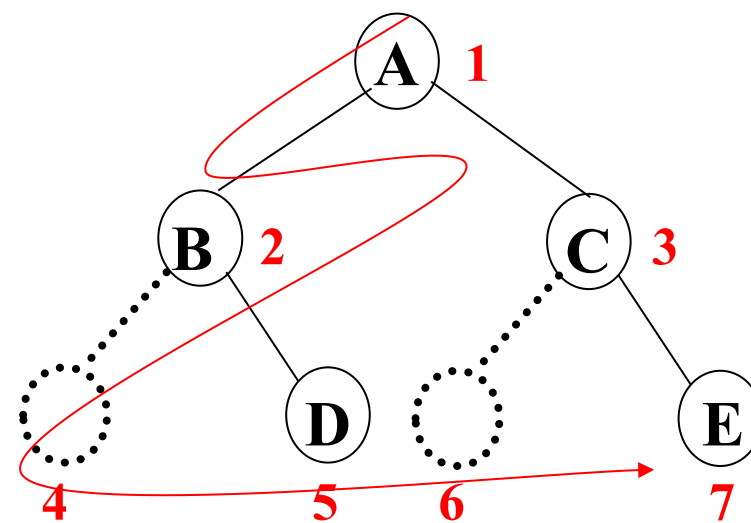
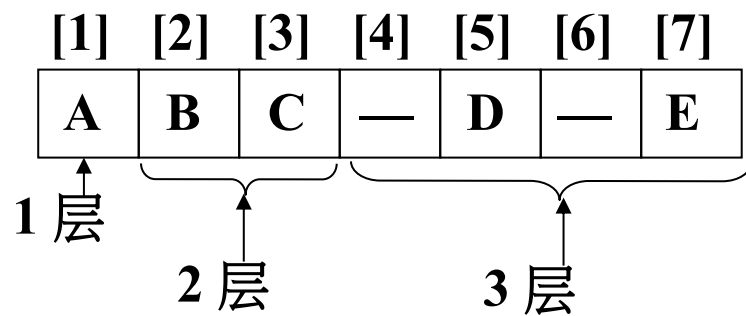
5 的左子节点 ?

5 的右子节点 ?

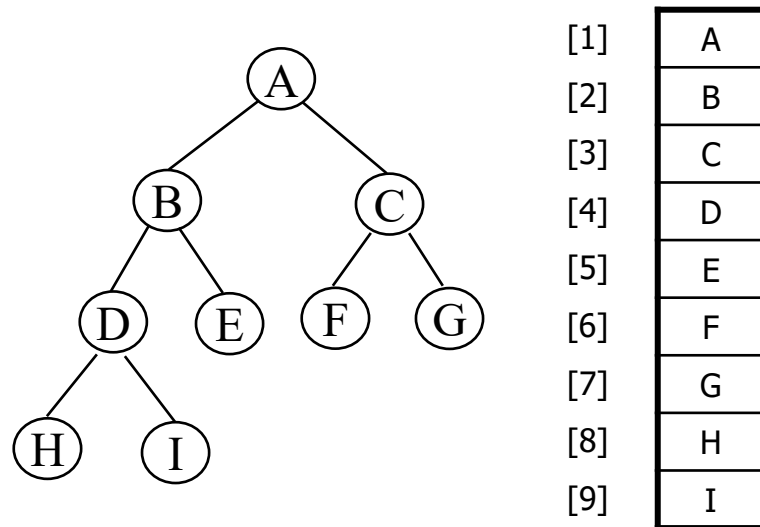
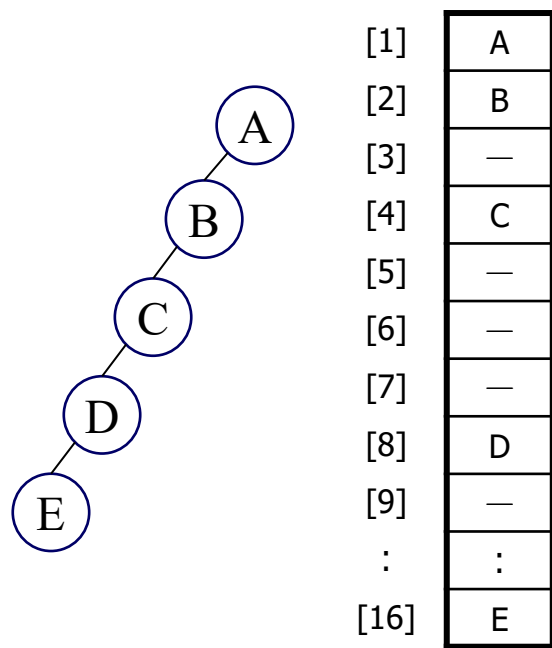
13 的左子节点 ?

13 的右子节点 ?

# 数组表示法



# 数组表示法

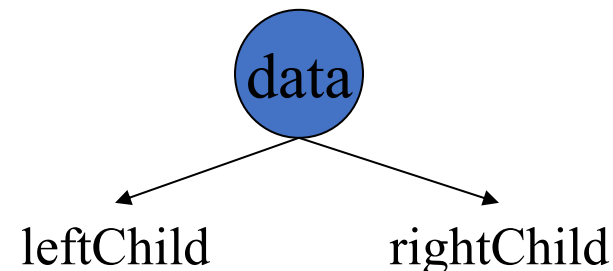


- ❖ 浪费空间: 最糟糕的情形, 一个深度为  $k$  的倾斜树将需要  $2^k-1$  个存储空间. 但是, 只有  $k$  个空间被利用。
- ❖ 从树的中间插入或删除节点需要移动潜在的大量节点, 以反映这些节点层级编号的变化。

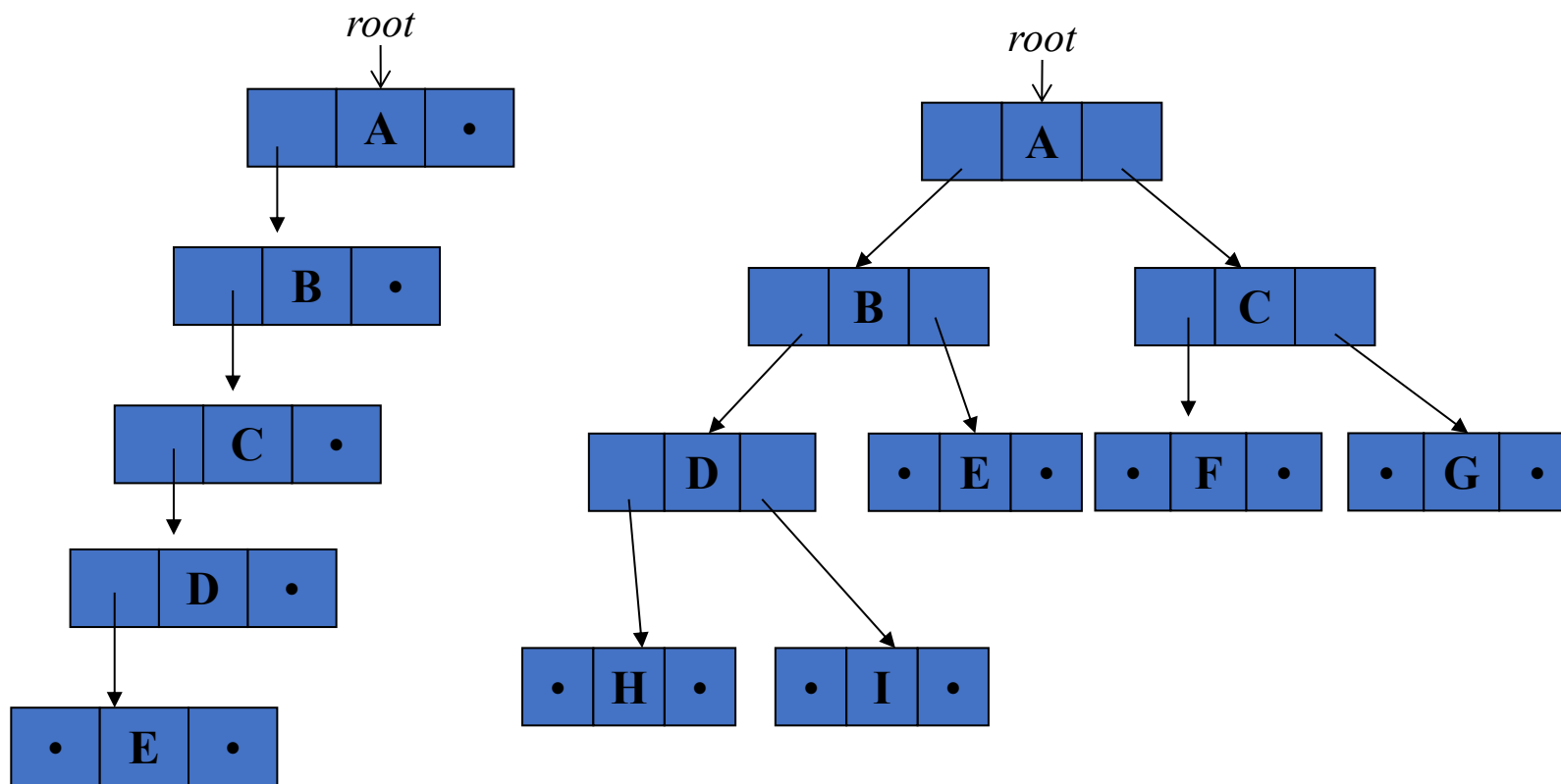
# 链表表示法

- ❖ 虽然数组表示法适合完全二叉树，但对于许多其他类型的二叉树来说会造成浪费。
  - 这个问题可以通过使用链表表示法来解决。
- ❖ 每个节点包含三个字段：左子节点（leftChild）、数据（data）和右子节点（rightChild）。

```
typedef struct node *treePointer;  
typedef struct node {  
    int data;  
    treePointer leftChild, rightChild;  
};
```



# 链表表示法示例



# 二叉树遍历

❖ 如何遍历一棵树或确保每个节点恰好被访问一次？

- 设 $L$ 、 $V$ 、 $R$ 分别表示向左移动、访问节点和向右移动。
- 遍历有6中可能的组合：

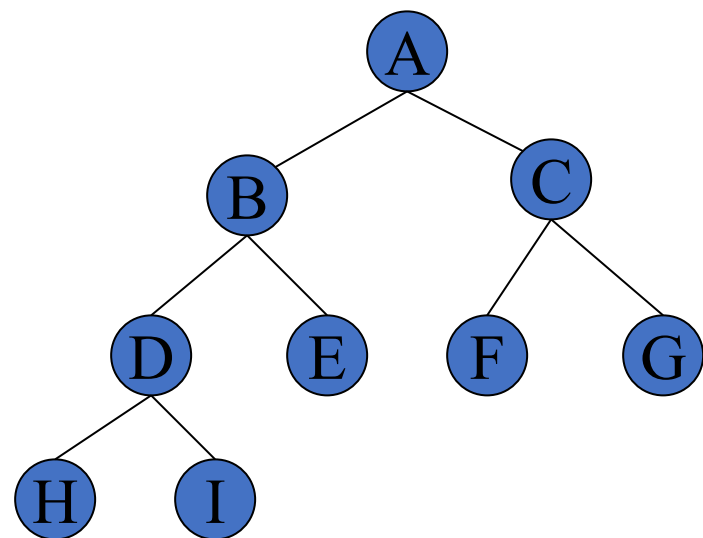
$LVR, LRV, VLR, VRL, RVL, RLV$

- 采用“先左后右”的惯例后，仅剩3种遍历方式：

$LVR$  (中序遍历),  $LRV$  (后序遍历),  $VLR$  (先序遍历)



# 二叉树遍历



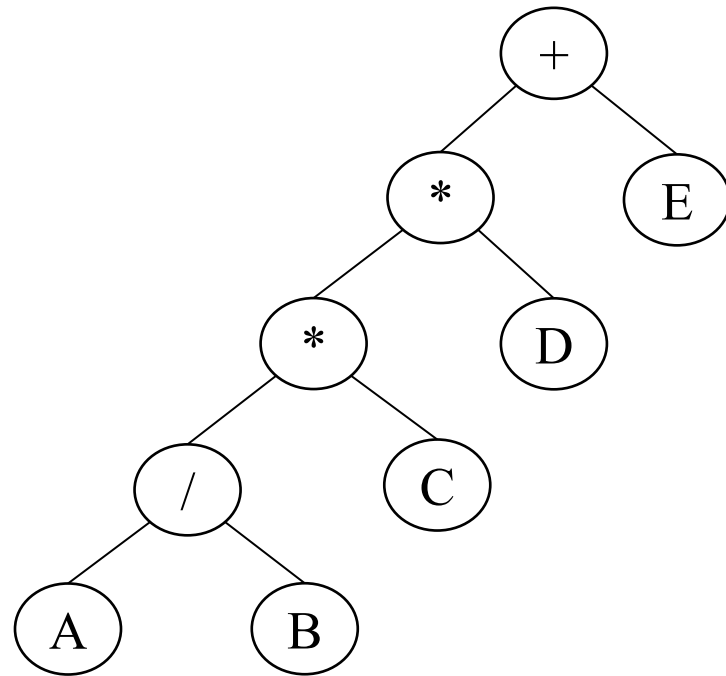
中序遍历 : H - D - I - B - E - A - F - C - G

先序遍历: A - B - D - H - I - E - C - F - G

后序遍历: H - I - D - E - B - F - G - C - A

# 使用二叉树表示的算术表达式

❖ 这些遍历方式与生成表达式的中缀、后缀和前缀形式之间存在自然的对应关系。



❖ 中序遍历

$A / B * C * D + E$   
(中缀表达式)

❖ 先序遍历

$+ * * / A B C D E$   
(前缀表达式)

❖ 后序遍历

$A B / C * D * E +$   
(后缀表达式)

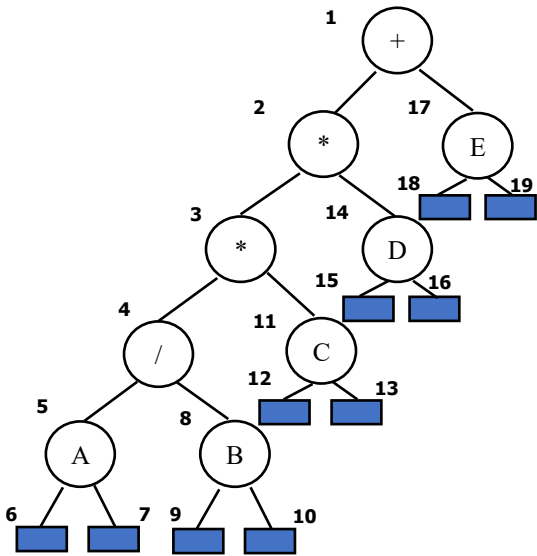
# 中序遍历

## ❖ 通俗解释，中序遍历要求

- 沿着树一直向左下去，直到无法再前进
- 然后“访问”该节点，向右移动一个节点并继续
- 如果无法向右移动，则回退一个节点

## ❖ 通过递归实现

```
void inorder(treePointer ptr)
{
    /* inorder tree traversal */
    if (ptr) {
        inorder(ptr->leftChild);
        printf("%d", ptr->data);
        inorder(ptr->rightChild);
    }
}
```



output:A / B \* C \* D + E

| Call of<br>inorder | Value<br>in root | Action | inorder | in root | Value<br>Action |
|--------------------|------------------|--------|---------|---------|-----------------|
| 1                  | +                |        | 11      | C       |                 |
| 2                  | *                |        | 12      | NULL    |                 |
| 3                  | *                |        | 11      | C       | printf          |
| 4                  | /                |        | 13      | NULL    |                 |
| 5                  | A                |        | 2       | *       | printf          |
| 6                  | NULL             |        | 14      | D       |                 |
| 5                  | A                | printf | 15      | NULL    |                 |
| 7                  | NULL             |        | 14      | D       | printf          |
| 4                  | /                | printf | 16      | NULL    |                 |
| 8                  | B                |        | 1       | +       | printf          |
| 9                  | NULL             |        | 17      | E       |                 |
| 8                  | B                | printf | 18      | NULL    |                 |
| 10                 | NULL             |        | 17      | E       | printf          |
| 3                  | *                | printf | 19      | NULL    |                 |

Figure 5.17

# 案例

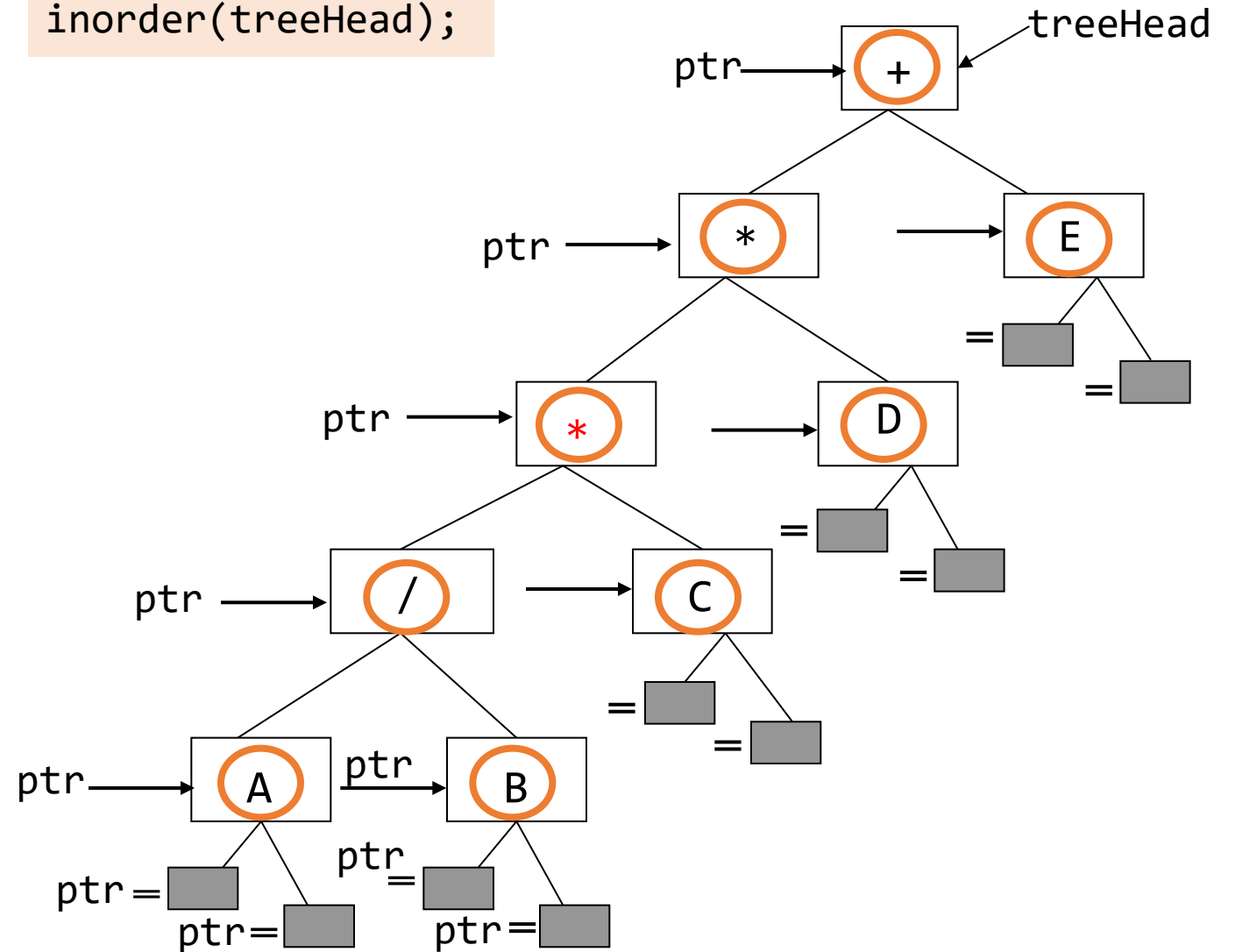
```
void inorder(treePointer ptr)
{
    if (ptr) {
        inorder(ptr->leftChild);
        printf("%c", ptr->data);
        inorder(ptr->rightChild);
    }
    return;
}
```

A / B \* C \* D + E

|                                                                                               |
|-----------------------------------------------------------------------------------------------|
| inorder(  )  |
| inorder(  )  |
| inorder(  )  |
| inorder(  ) |
| inorder( E )                                                                                  |
| inorder( + )                                                                                  |
| main()                                                                                        |

函数调用栈

inorder(treeHead);

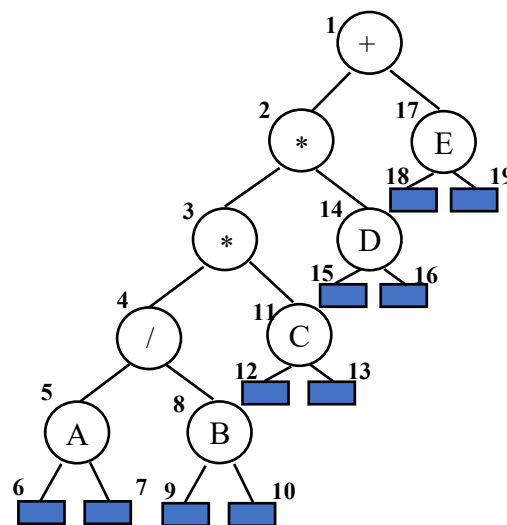


# 先序遍历

- ❖ 访问一个节点，向左遍历并继续。
- ❖ 当无法继续时，享有移动并重新开始
- ❖ 或者回退，知道可以向右移动并恢复

```
void preorder(treePointer ptr)
{   /* preorder tree traversal */
    if (ptr) {
        printf("%d", ptr->data);
        preorder(ptr->leftChild);
        preorder(ptr->rightChild);
    }
}
```

output: + \* \* / A B C D E

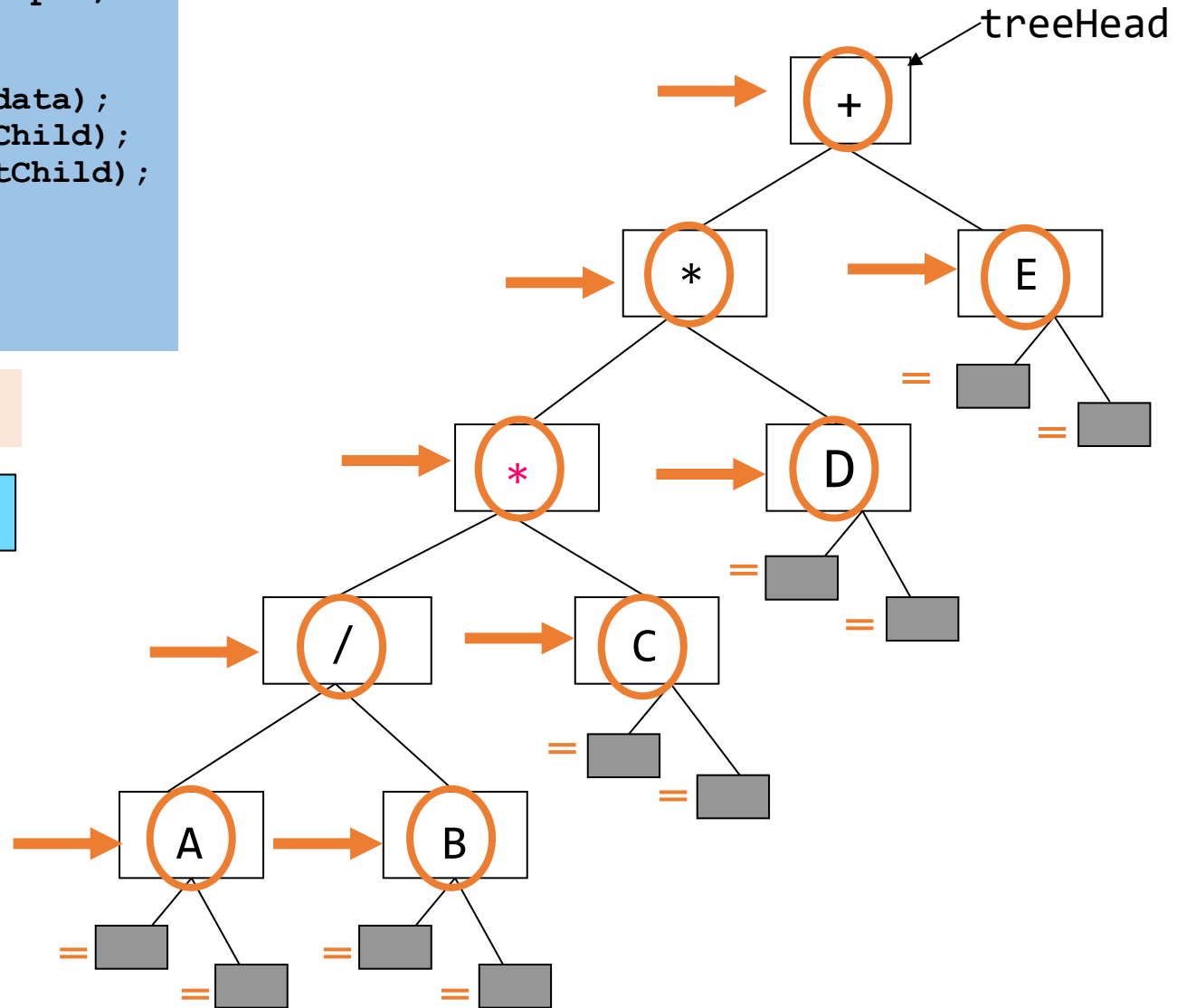


## 案例

```
void preorder(treePointer ptr)
{
    if (ptr) {
        printf("%c", ptr->data);
        preorder(ptr->leftChild);
        preorder(ptr->rightChild);
    }
    return;
}
```

preorder(treeHead);

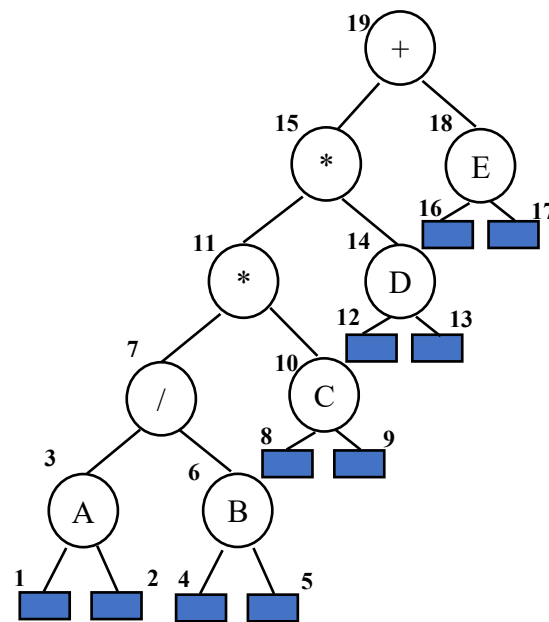
+ \* \* / A B C D E



# 后序遍历

```
void postorder(treePointer ptr)
{   /* postorder tree traversal */
    if (ptr) {
        postorder(ptr->leftChild);
        postorder(ptr->rightChild);
        printf("%d", ptr->data);
    }
}
```

output: A B / C \* D \* E +

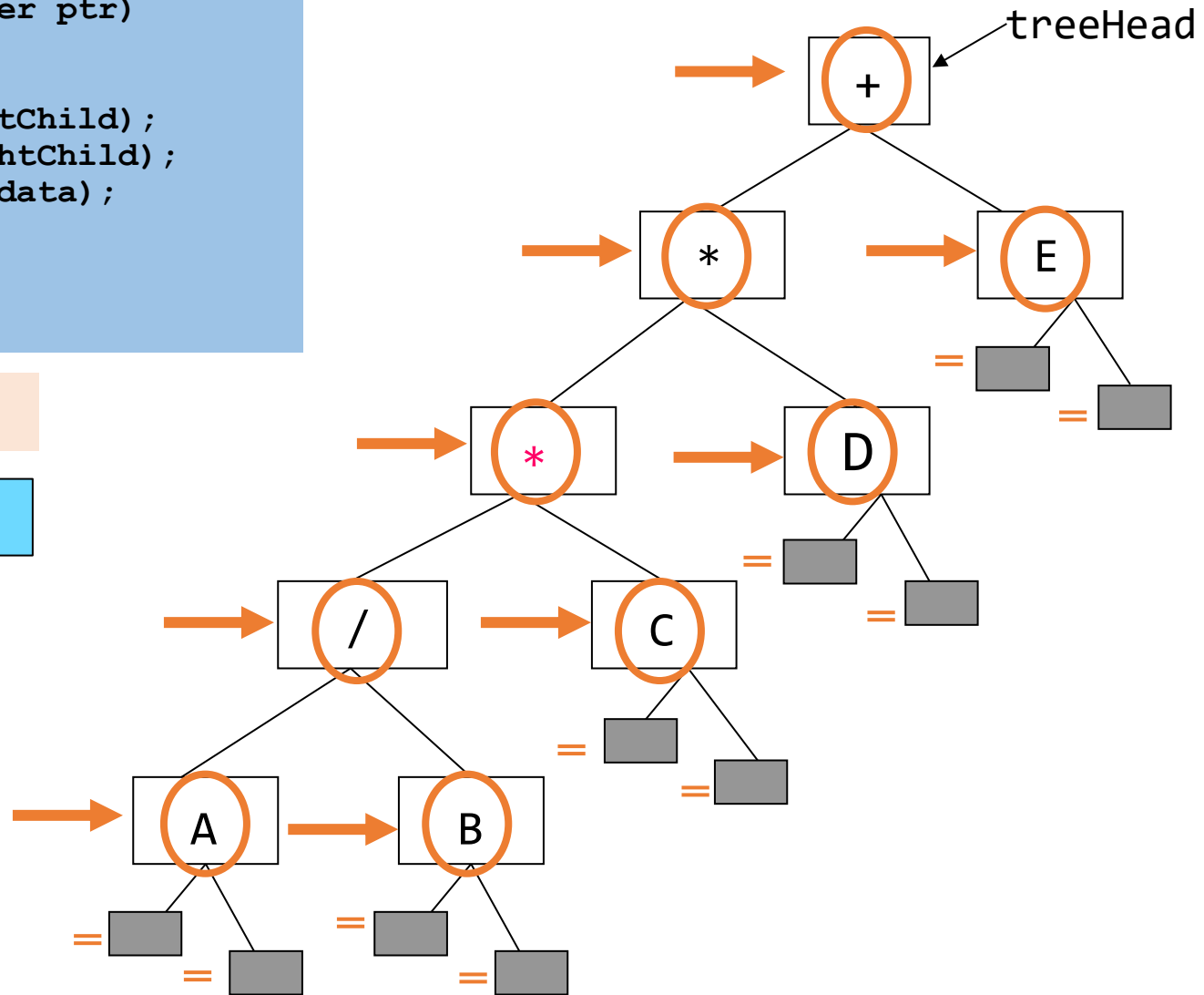


## 案例

```
void postorder(treePointer ptr)
{
    if (ptr) {
        postorder(ptr->leftChild);
        postorder(ptr->rightChild);
        printf("%c", ptr->data);
    }
    return;
}
```

postorder(treeHead);

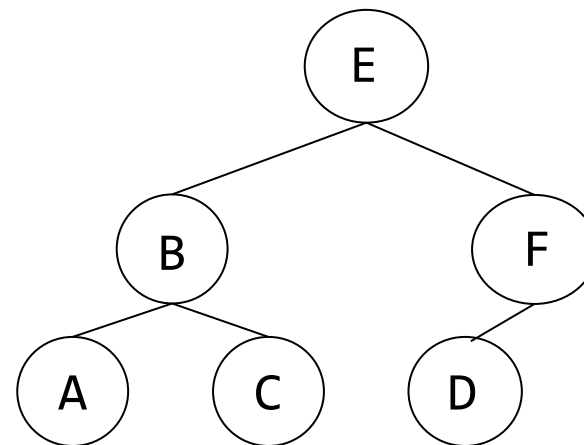
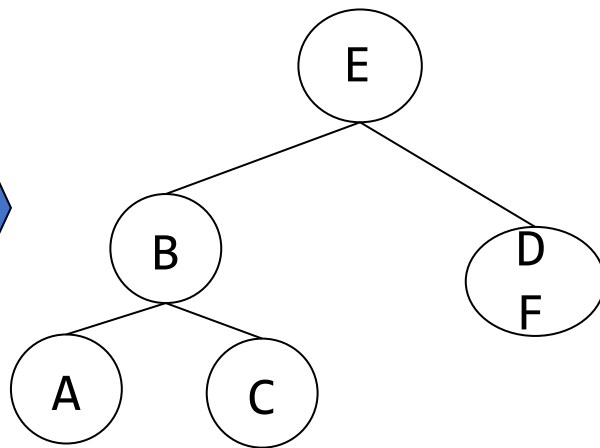
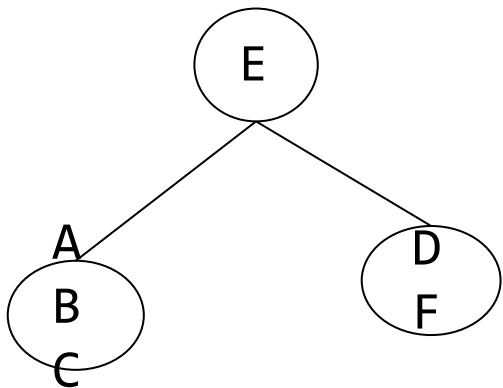
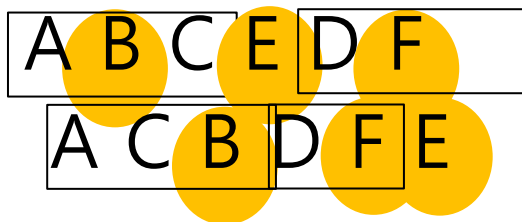
A B / C \* D \* E +





# 如何从遍历结果重建一棵树

中序遍历  
后序遍历



# 迭代中序遍历

❖ 可以使用栈开发与中序、前序和后序遍历函数等价的迭代函数

- 图 5.17 展示了压栈和出栈的过程
- 一个没有动作的节点表示该节点被加入栈中。
  - 一个带有 printf 动作的节点表示该节点从栈中移除。

```
void iterInorder(treePointer node)
{
    int top= -1; /* initialize stack */
    treePointer stack[MAX_STACK_SIZE];
    for ( ; ; ) {
        for ( ; node; node=node->leftChild )
            push(node); /* add to stack */
        node = pop(); /* delete from stack */
        if (!node) break; /* empty stack */
        printf("%d", node->data);
        node = node->rightChild;
    }
}
```

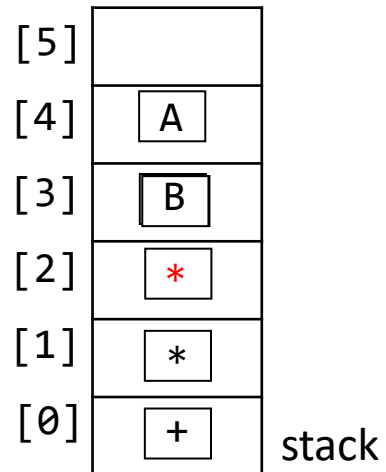
| Call of<br><i>inorder</i> | Value<br>in root | Action | <i>inorder</i> | in root | Value<br>Action |
|---------------------------|------------------|--------|----------------|---------|-----------------|
| 1                         | +                |        | 11             | C       |                 |
| 2                         | *                |        | 12             | NULL    |                 |
| 3                         | *                |        | 11             | C       | printf          |
| 4                         | /                |        | 13             | NULL    |                 |
| 5                         | A                |        | 2              | *       | printf          |
| 6                         | NULL             |        | 14             | D       |                 |
| 5                         | A                | printf | 15             | NULL    |                 |
| 7                         | NULL             |        | 14             | D       | printf          |
| 4                         | /                | printf | 16             | NULL    |                 |
| 8                         | B                |        | 1              | +       | printf          |
| 9                         | NULL             |        | 17             | E       |                 |
| 8                         | B                | printf | 18             | NULL    |                 |
| 10                        | NULL             |        | 17             | E       | printf          |
| 3                         | *                | printf | 19             | NULL    |                 |

Figure 5.17

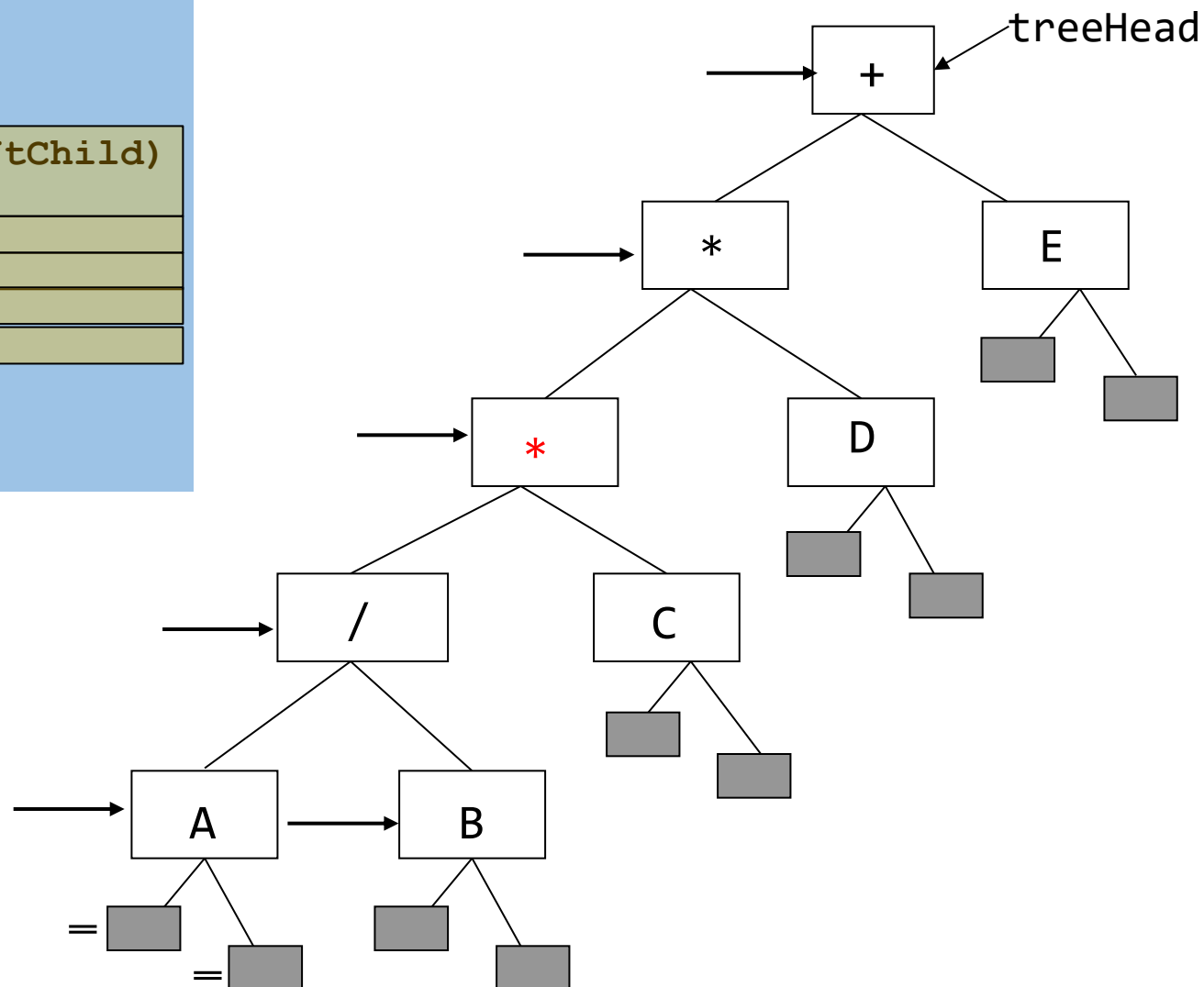
# 案例

```
int top = -1;
treePointer stack[MAX_STACK_SIZE];
void iterInorder(treePointer node)
{
    for (;;) {
        for (; node; node = node->leftChild)
            push(node);
        node = pop();
        if (!node) break;
        printf("%c", node->data);
        node = node->rightChild;
    }
    return;
}
```

A /

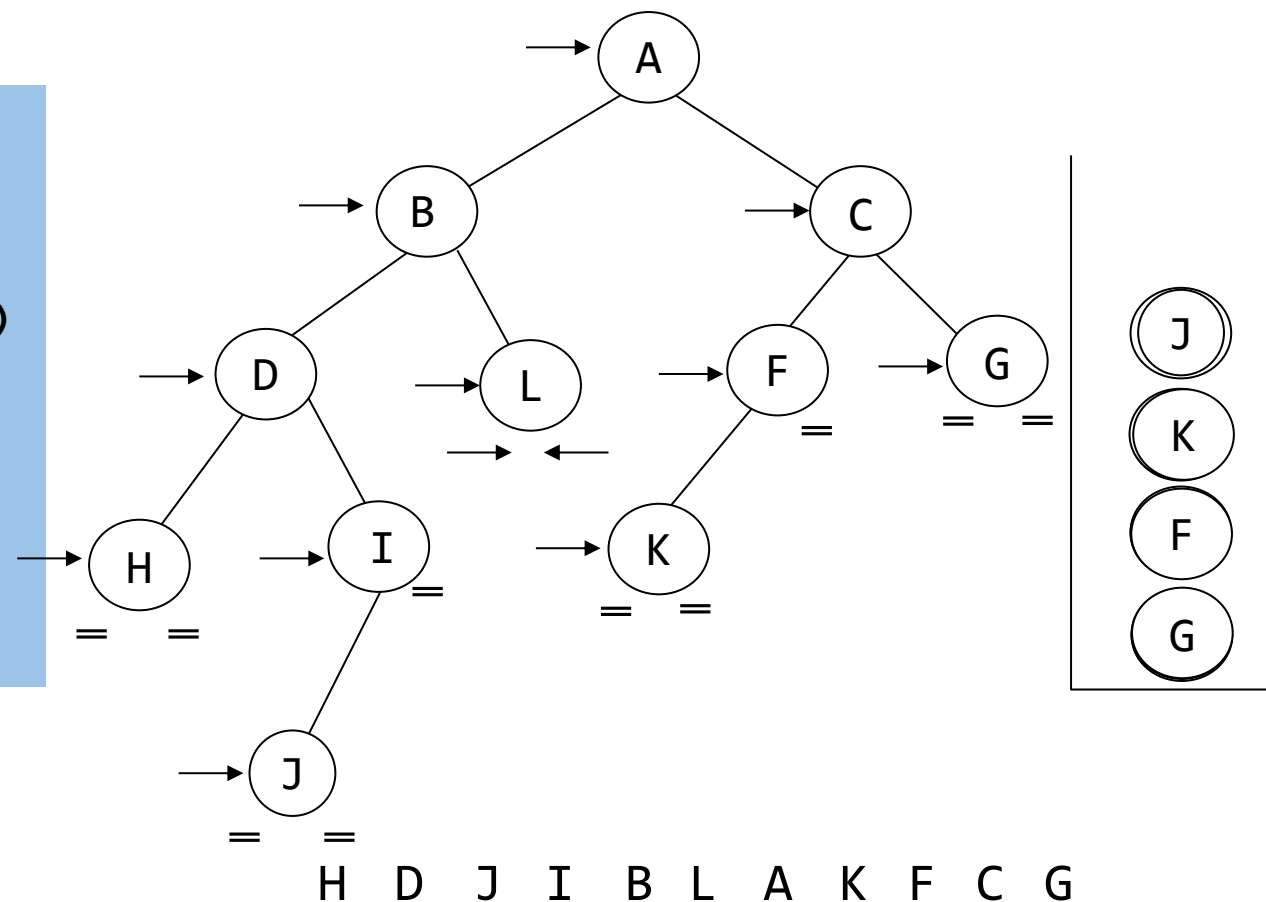


iterInorder(treeHead);



## 案例

```
int top = -1;
treePointer stack[MAX_STACK_SIZE];
void iterInorder(treePointer node)
{
    for (;;) {
        for (; node; node = node->leftChild)
            push(node);
        node = pop();
        if (!node) break;
        printf("%c", node->data);
        node = node->rightChild;
    }
    return;
}
```

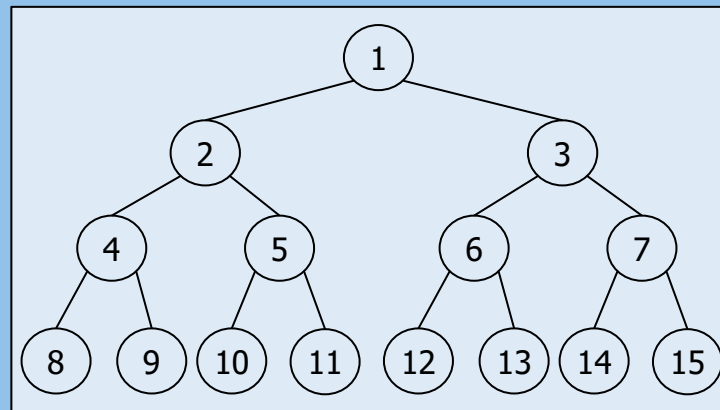


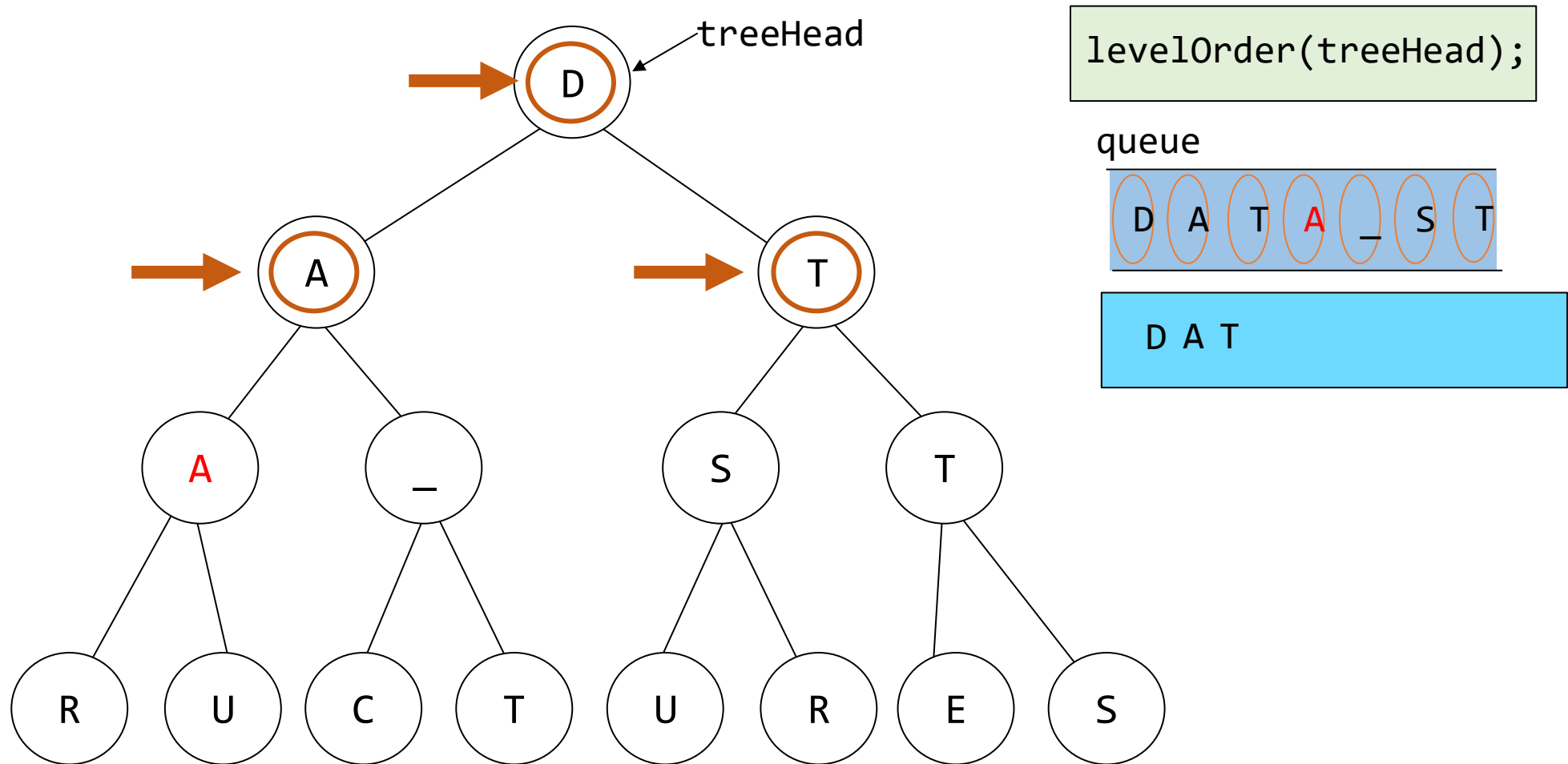
# 层次遍历 (=广度优先遍历)

## ❖ 需要队列的遍历

- 先访问根节点，然后是根的左子节点，接着是跟的右子节点

```
void levelOrder(treePointer ptr)
{/* level order tree traversal */
    int front = rear = 0;
    treePointer queue[MAX_QUEUE_SIZE];
    if (!ptr) return; /* empty tree */
    addq( ptr );
    for ( ; ; ) {
        ptr = deleteq();
        if (ptr) {
            printf("%d", ptr->data);
            if (ptr->leftChild)
                addq(ptr->leftChild);
            if (ptr->rightChild)
                addq(ptr->rightChild);
        }
        else break;
    }
}
```





# 二叉树的其他操作

## ❖ 复制二叉树

- 使用稍作修改后的后序遍历算法

```
treePointer copy(treePointer original)
{
    /* this function returns a treePointer to an exact copy of the original tree */
    treePointer temp;
    if (original)
    {
        temp = MALLOC( sizeof(*temp) );
        temp->leftChild = copy(original->leftChild);
        temp->rightChild = copy(original->rightChild);
        temp->data = original->data;
        return temp;
    }
    return NULL;
}
```

# 测试相等性

| A     | B     | A  B  |
|-------|-------|-------|
| True  | True  | True  |
| True  | False | True  |
| False | True  | True  |
| False | False | False |

## ❖ 两棵二叉树的等价性

- 如果两棵二叉树具有相同的结构，并且对应节点中的信息相同，则它们是等价的。

```
int equal( treePointer first, treePointer second )
{
    /* function returns FALSE if the binary trees first and second are not equal,
    otherwise it returns TRUE */

    return ( (!first && !second) || (first && second &&
        (first->data == second->data) &&
        equal(first->leftChild, second->leftChild) &&
        equal(first->rightChild, second->rightChild)) );    /*return (A||B);
}
}
```



# 线索二叉树

❖ 在任何二叉树的链表表示中，空链接（null links）的数量多于实际指针

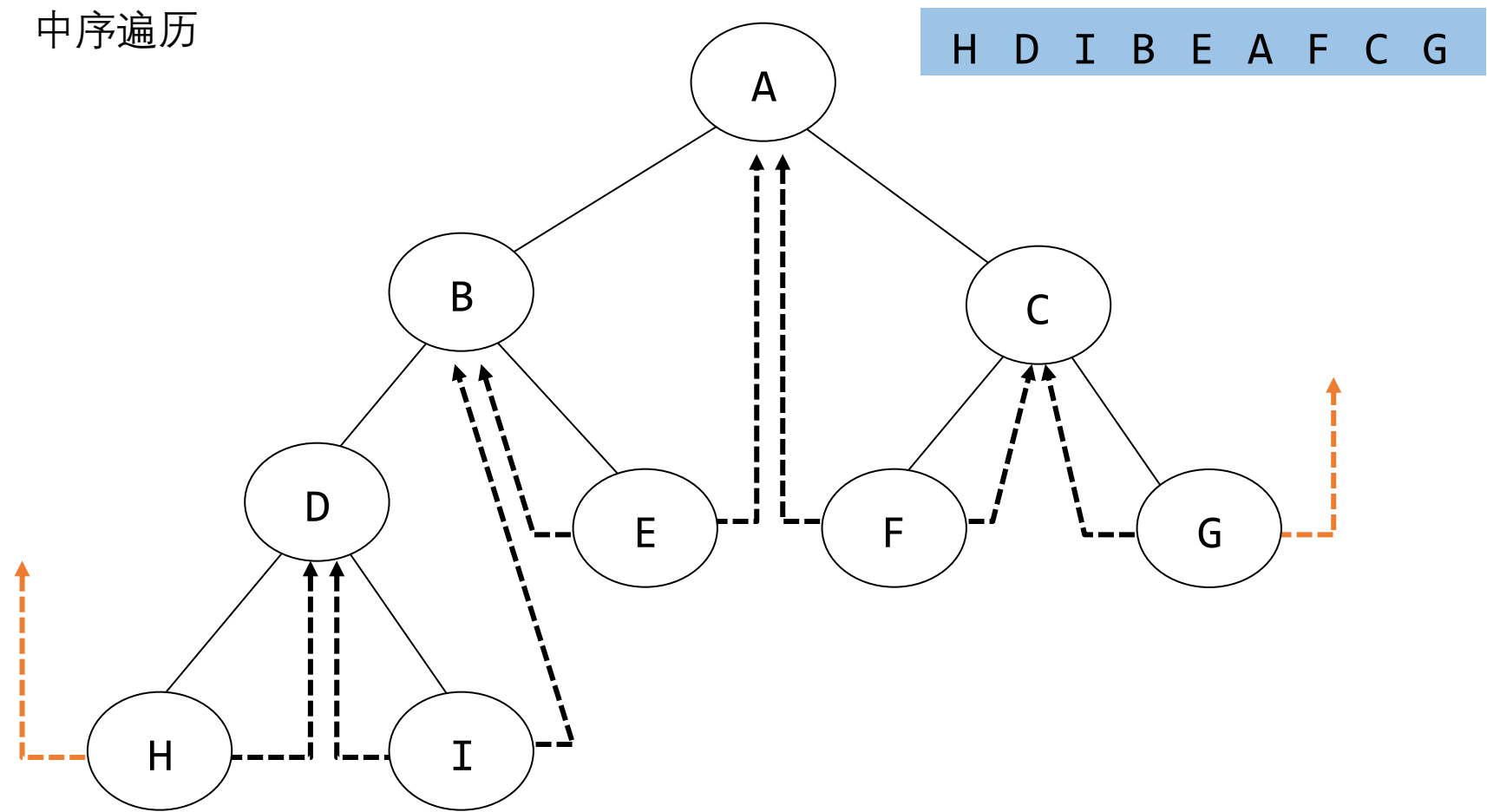
- $n$ ：节点数量
- 非空链接数量： $n-1$
- 总链接数量： $2n$
- 空链接数量： $2n-(n-1) = n+1$
- 将空链接替换为指向其他节点的指针，称为“线索”（threads）

❖ 线索树构造规则

- If `ptr->leftChild` is null
  - 将其替换为指向在中序遍历中 *ptr* 之前访问的节点的指针，即 *ptr* 的中序前驱
- If `ptr->right_child` is null
  - 将其替换为指向再中序遍历中 *ptr* 之后访问的节点的指针，即 *ptr* 的中序后继

# 线索二叉树案例

中序遍历

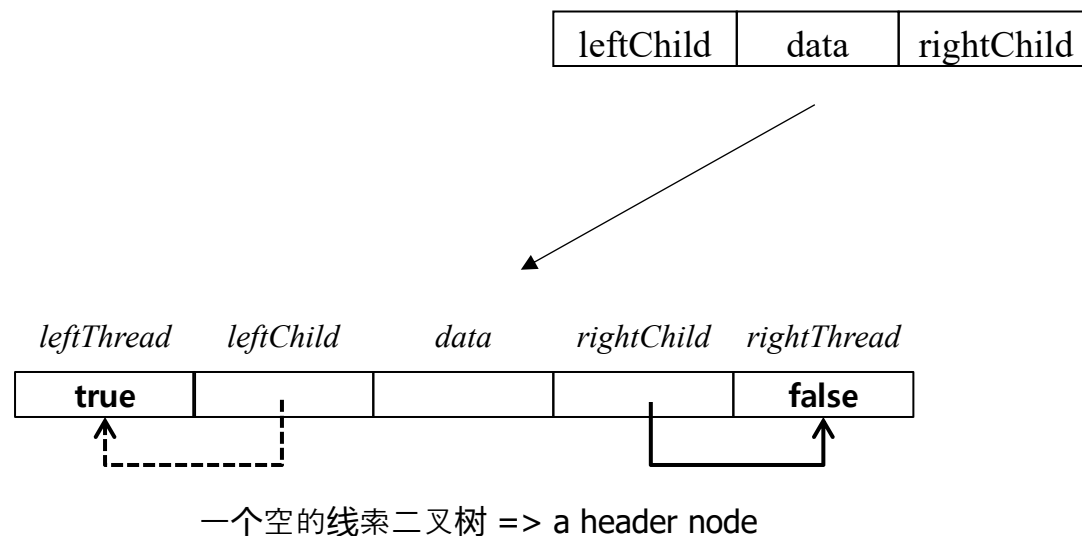


# 线索二叉树

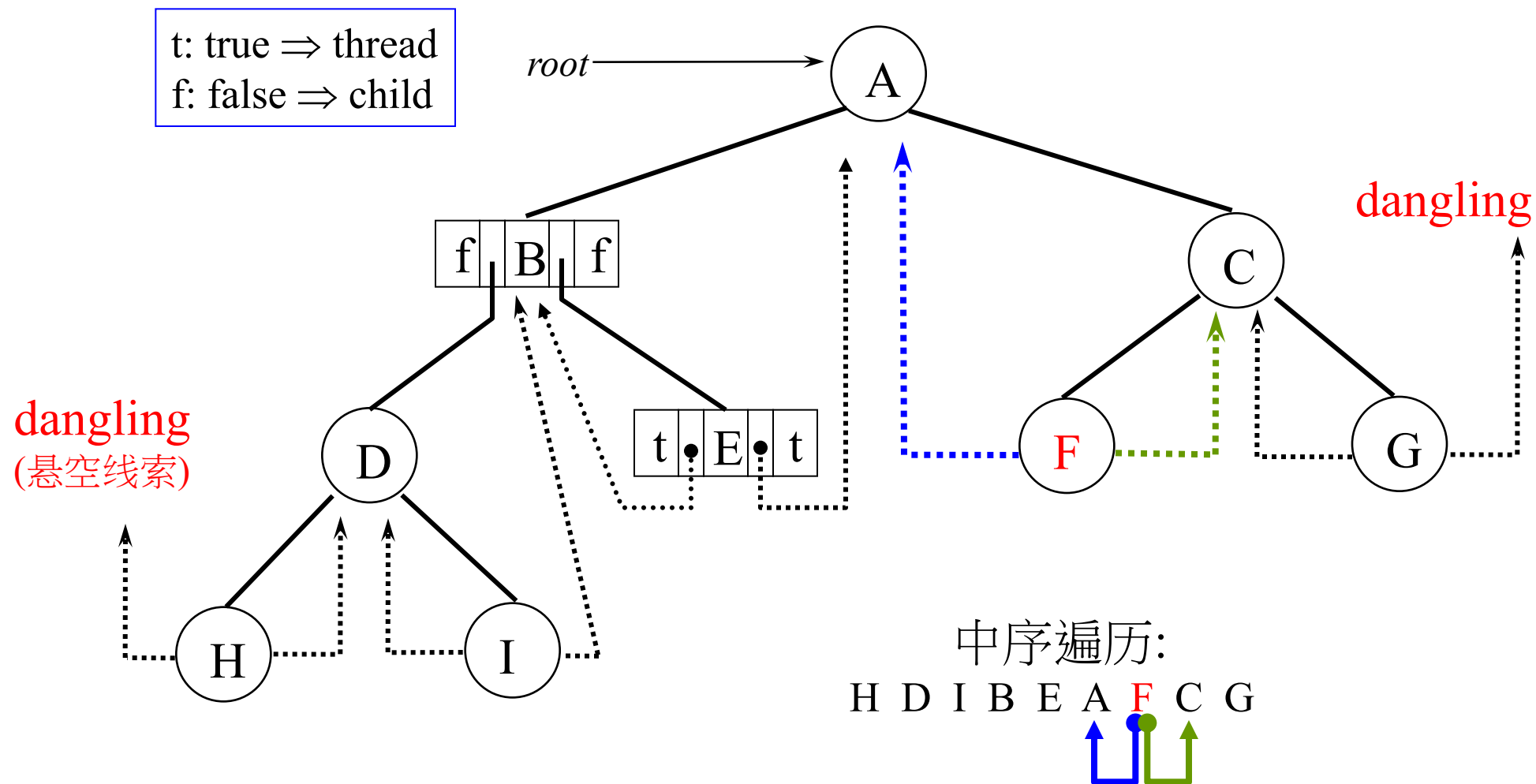
## • 节点结构中的两个附加字段

- leftThread 与 rightThread
- If ptr->leftThread=TRUE
  - ptr->leftChild 包含一个线索
  - 否则，它包含指向左子节点的指针
- ptr->rightThread 同理

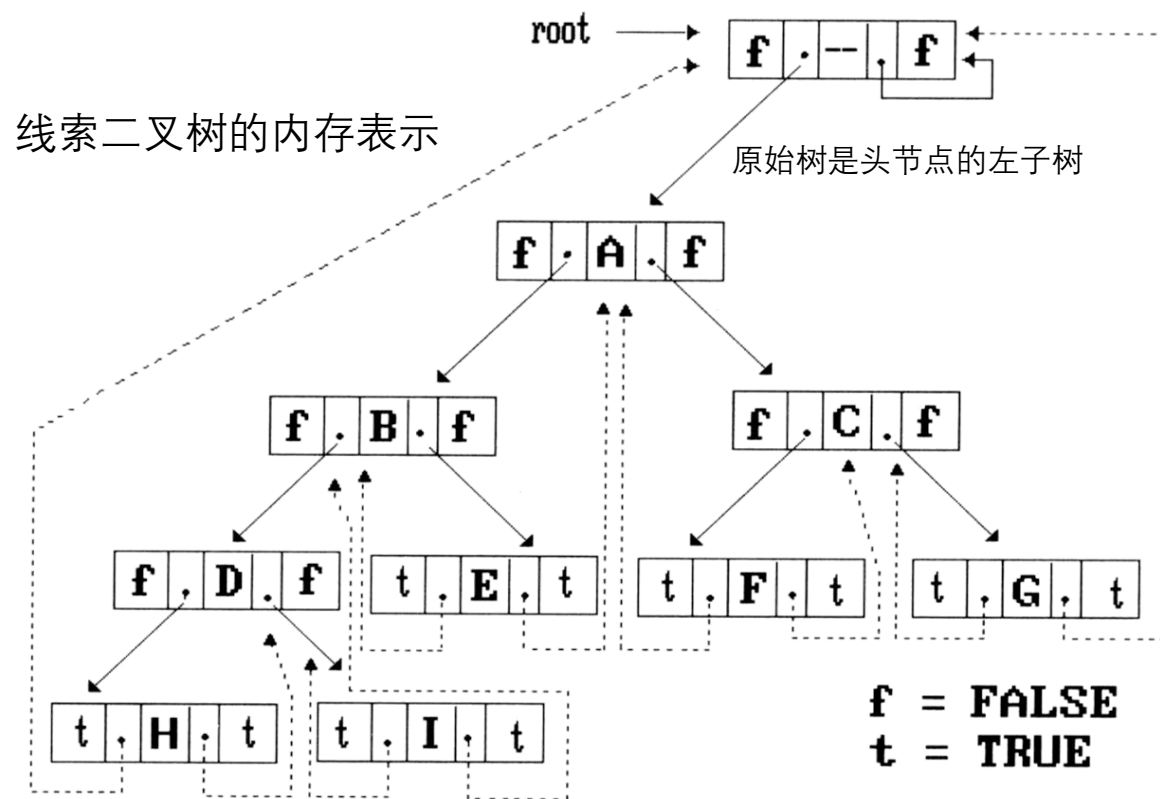
```
typedef struct threadedTree *threadedPointer;  
typedef struct threadedTree {  
    short int leftThread;  
    threadedPointer leftChild;  
    char data;  
    threadedPointer rightChild;  
    short int rightThread;  
};
```



# 线索二叉树



# 线索二叉树



## ❖ 避免出现悬空指针

- 头节点的左指针(H的左指针)和G的右指针
- 创建一个根节点，并使这些指针指向该根节点

# 线索二叉树

## ❖ 线索二叉树的中序遍历

- 我们可以在不使用栈的情况下进行中序遍历

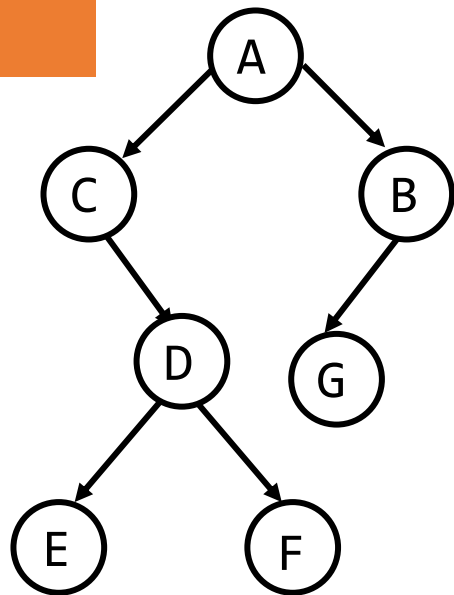
1. If  $ptr \rightarrow rightThread = TRUE$ ,  $ptr$  的中序后继是  $ptr \rightarrow rightChild$
2. 否则, 从  $ptr$  的右子节点开始, 沿着左子节点链接向下走, 直到遇到一个  $leftThread = TRUE$  的节点

```
void tinorder( threadedPointer tree ) {    /* traverse the threaded binary tree inorder */
    threadedPointer temp = tree;
    for( ; ; ) {
        temp = insucc( temp );
        if( temp == tree ) break;
        printf( "%3c", temp->data );
    }
}
```

```
threadedPointer insucc(threadedPointer tree)
{
    /*find the inorder successor of tree in a threaded binary tree */
    threadedPointer temp;
    temp = tree->rightChild;
    if( !tree->rightThread )
        while( !temp->leftThread )
            temp = temp->leftChild;

    return temp;
}
```

## 案例



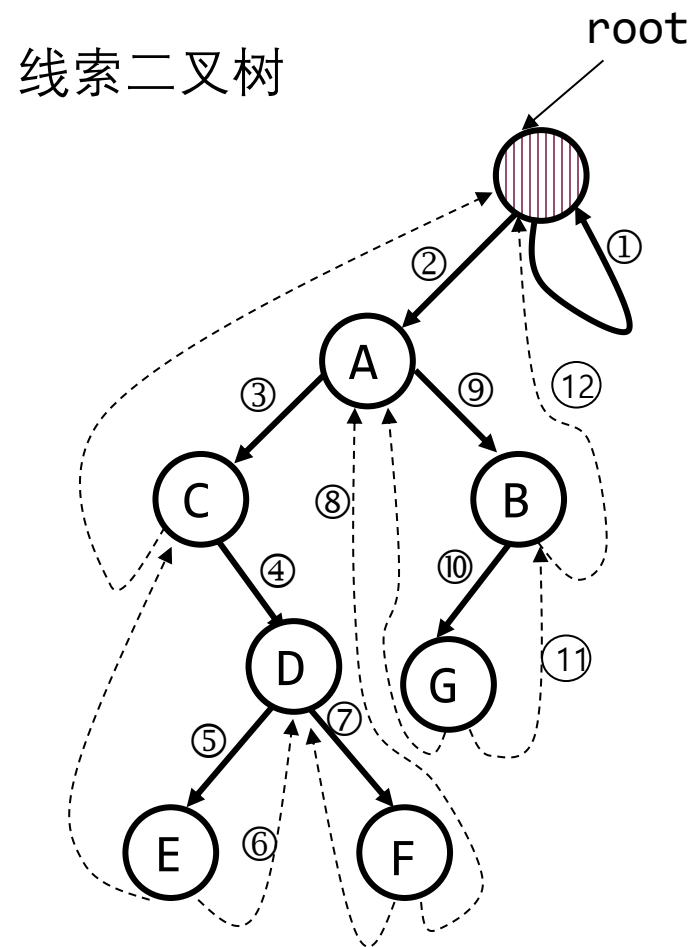
中序遍历结果  
C E D F A G B

```
void tinorder( threadedPointer tree ) {    /* traverse the threaded binary tree inorder */
    threadedPointer temp = tree;
    for( ; ; ) {
        temp = insucc( temp );
        if( temp == tree ) break;
        printf( "%3c", temp->data );
    }
}
```

```

threadedPointer insucc(threadedPointer tree)
{
    /*find the inorder successor of tree in a threaded binary tree */
    threadedPointer temp;
    temp = tree->rightChild;
    if( !tree->rightThread )
        while( !temp->leftThread )
            temp = temp->leftChild;
    return temp;
}

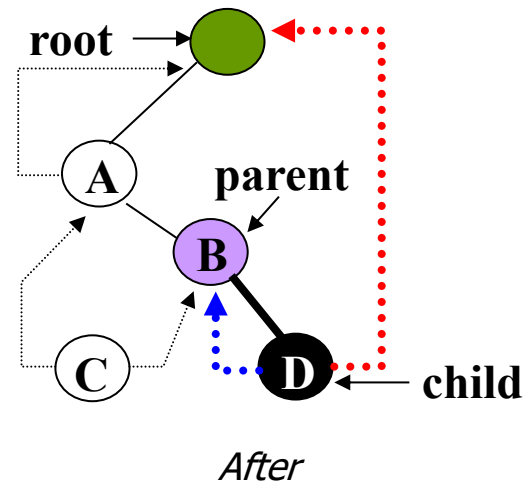
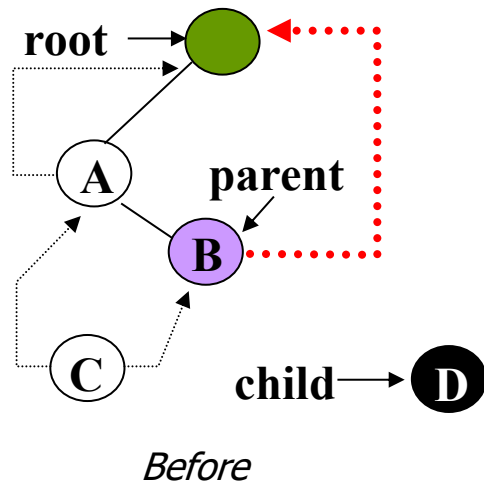
```



# 线索二叉树

## ❖ 向线索二叉树中插入一个节点

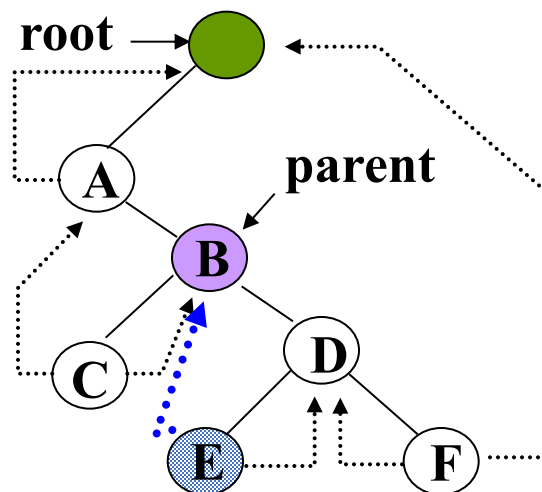
- 将D作为B的右子节点插入
- 如果B右子树为空，那么插入操作简单





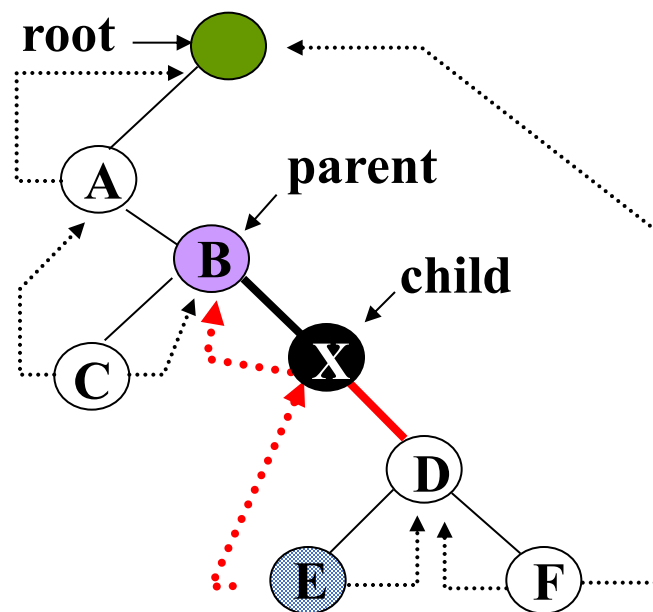
# 线索二叉树

❖ 在线索二叉树中插入父节点的右子节点



child → **X**

*Before*



*After*

# 将一个节点插入作为右子节点

```
void insertRight(threadedPointer s, threadedPointer r)
```

```
/* 将 r 作为 s 的右子节点插入 */
```

```
{
```

```
    threadedPointer temp;
```

```
    r->rightChild = s->rightChild;
```

```
    r->rightThread = s->rightThread;
```

```
    r->leftChild = s;
```

```
    r->leftThread = TRUE;
```

```
    s->rightChild = r;
```

```
    s->rightThread = FALSE;
```

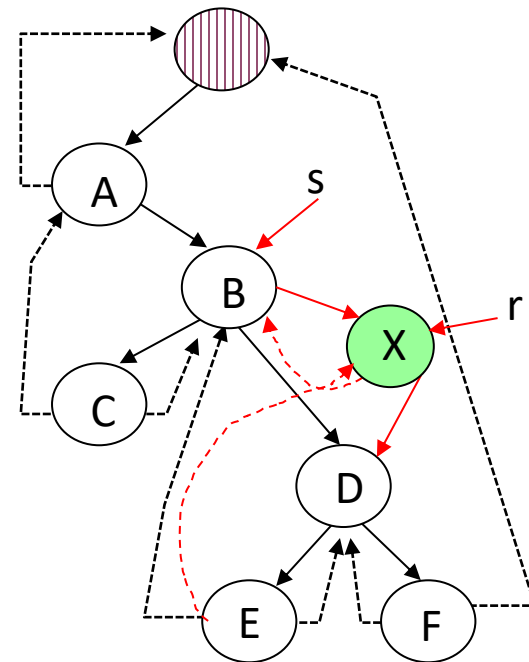
```
    if (!r->rightThread){
```

```
        temp = insucc(r);
```

```
        temp->leftChild = r;
```

```
    }
```

```
}
```



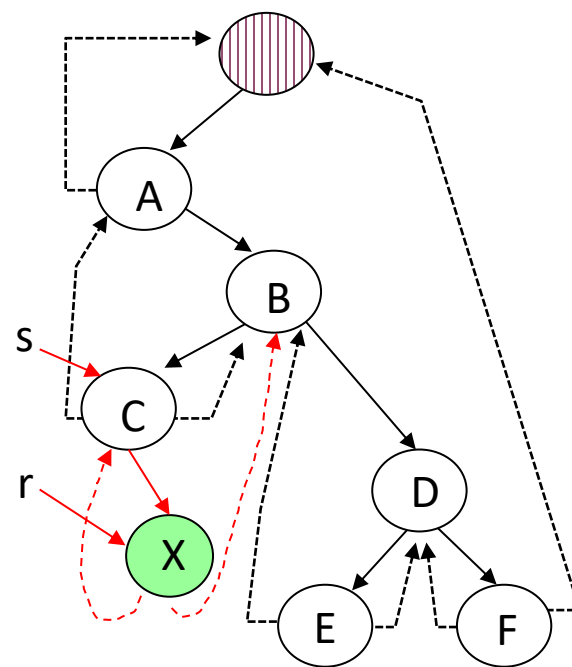
中序遍历

插入前 : A C B E D F

插入后 : A C B X E D F

# 将一个节点插入作为右子节点

```
void insertRight(threadedPointer s, threadedPointer r)
/* 将 r 作为 s 的右子节点插入 */
{
    threadedPointer temp;
    r->rightChild = s->rightChild;
    r->rightThread = s->rightThread;
    r->leftChild = s;
    r->leftThread = TRUE;
    s->rightChild = r;
    s->rightThread = FALSE;
    if (!r->rightThread){
        temp = insucc(r);
        temp->leftChild = r;
    }
}
```



# 堆

## ❖ 优先队列

- 堆常用于实现优先队列
- 要删除的元素是优先级最高(或最低)的元素

**ADT** *MaxPriorityQueue* is

**objects:** a collection of  $n > 0$  elements, each element has a key

**functions:**

for all  $q \in \text{MaxPriorityQueue}$ ,  $item \in \text{Element}$ ,  $n \in \text{integer}$

*MaxPriorityQueue* **create**( *max\_size* )                   ::= 创建一个空的优先队列

*Boolean* **isEmpty**(*q*, *n*)                   ::= **if**( $n > 0$ ) **return** FALSE

**else return** TRUE

*Element* **top**(*q*, *n*)                   ::= **if**( **!isEmpty**(*q*, *n*) ) **return** *q*中最大元素的实例

**else return** error

*Element* **pop**(*q*, *n*)                   ::= **if**( **!isEmpty**(*q*, *n*) ) **return** 找到堆 *q* 中的最大元素的一个实例并将其移除

**else return** error

*MaxPriorityQueue* **push**(*q*, *item*, *n*) ::= 将 *item* 插入 *q* , 并返回生成的优先级队列

# 堆

## ❖ 示例：销售机器的服务

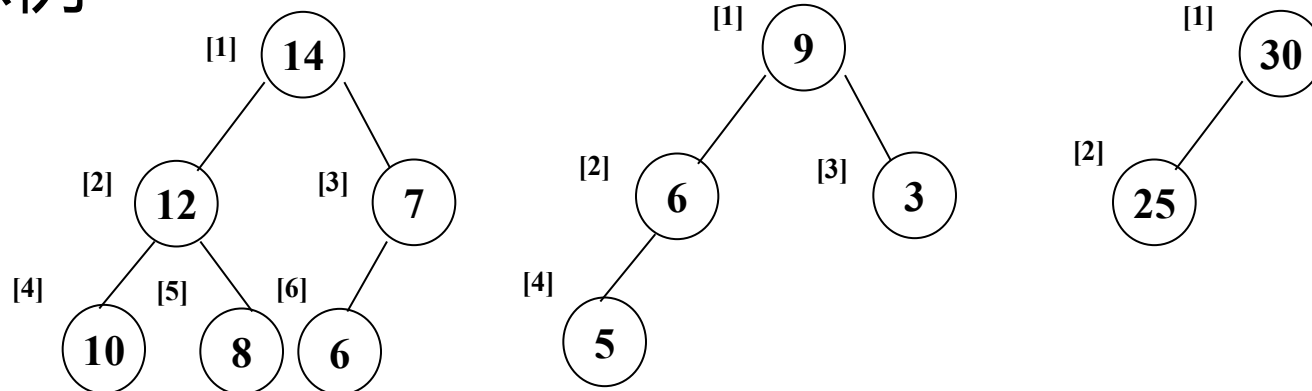
- 每个用户每次使用支付固定费用
- 每个用户所需的时间不同
- 如何在假设机器不能闲置(除非没有用户可用)的条件下最大化收益?
- 可以通过维护一个等待使用机器的所有人的优先级队列来实现
  - 选择所需时间最短的用户
  - 需要一个最小优先级队列

# 最大堆

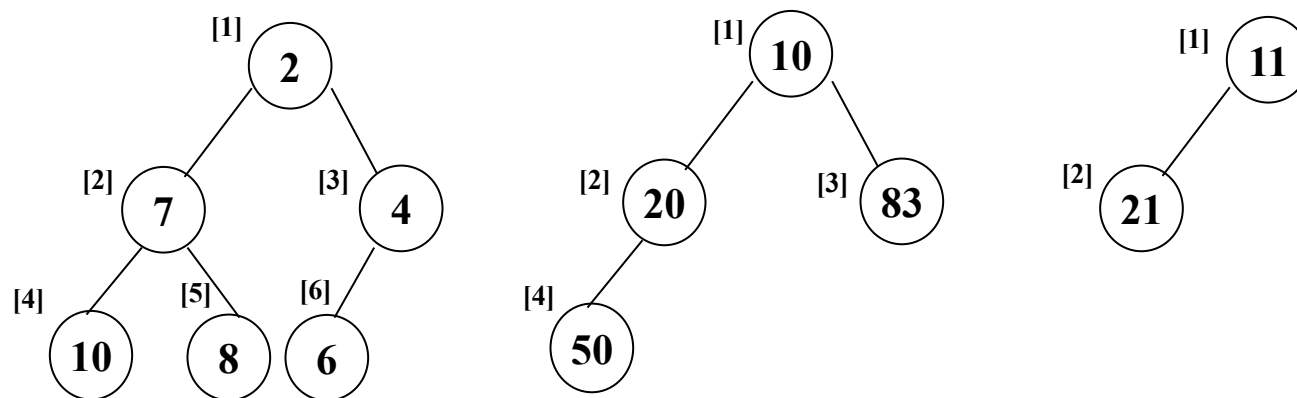
- ❖ 最大树：一棵树，其中每个节点的键值不小于其子节点(如果有)的键值
- ❖ 最大堆：一个完全二叉树，同时也是最大树
- ❖ 最小树：一棵树，其中每个节点的键值不大于其子节点(如果有)的键值
- ❖ 最小堆：一个完全二叉树，同时也是最小树
- ❖ 基本操作与最大优先级队列相同
- ❖ 由于最大堆是完全二叉树，我们使用数组堆来表示它

# 堆

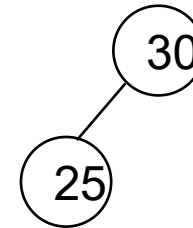
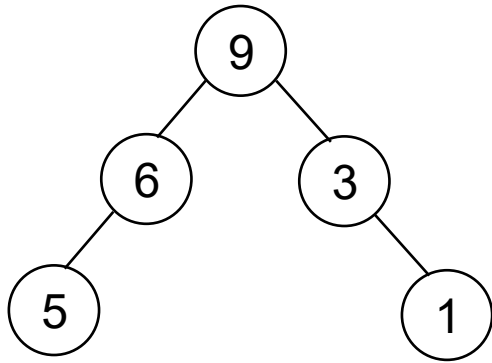
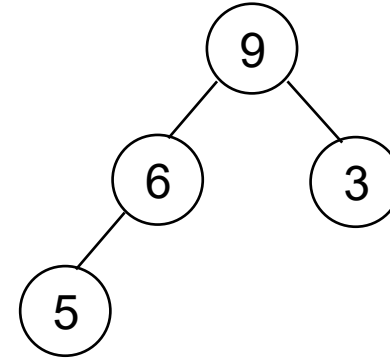
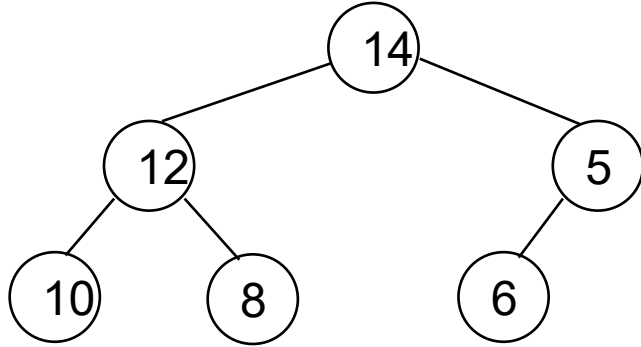
## ❖ 最大堆示例



## ❖ 最小堆示例

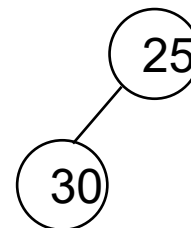
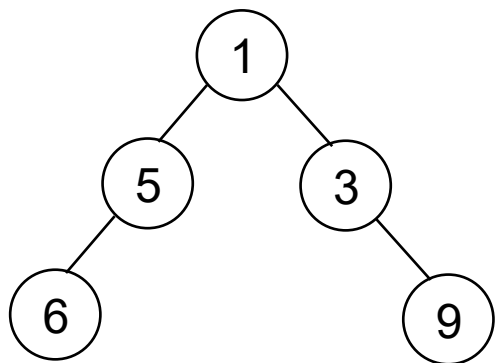
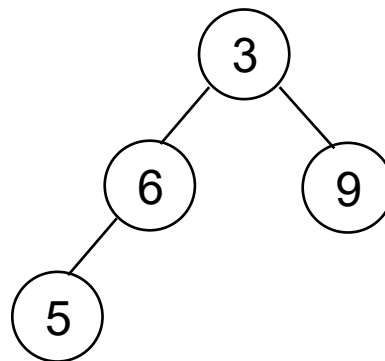
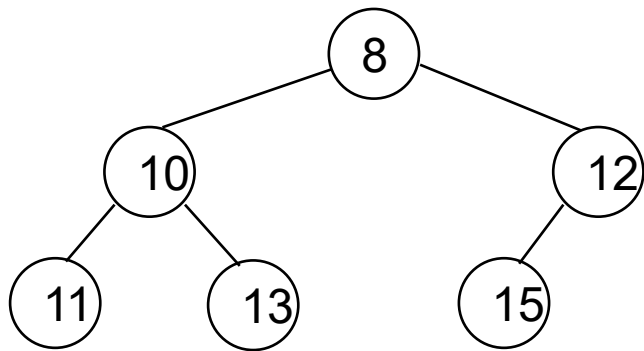


# 这些是最大堆吗？





# 这些是最小堆吗？



# 优先队列

## ❖ 优先队列

- 加入队列的数据元素带有与之相关的优先级（支付金额、重要性等）
- 一个队列，其中项按优先级排序，确保最高优先级的项始终是下一个被取出的

## ❖ 我们可以使用树(数据结构)

- 它通常提供插入和删除的  $O(\log n)$  性能( $n$ 是树中节点数)
- 不幸的是，如果树变得不平衡，在极端情况下性能会退化为 $O(n)$ 
  - 在处理时间关键的情况时，这可能无法接受

## ❖ 堆将为插入和删除提供保证的 $O(\log n)$ 性能

# 优先队列的表示方法

| 表示方法 | 插入            | 删除            |
|------|---------------|---------------|
| 无序数组 | $\theta(1)$   | $\theta(n)$   |
| 无序链表 | $\theta(1)$   | $\theta(n)$   |
| 有序数组 | $O(n)$        | $\theta(1)$   |
| 有序链表 | $O(n)$        | $\theta(1)$   |
| 最大堆  | $O(\log_2 n)$ | $O(\log_2 n)$ |

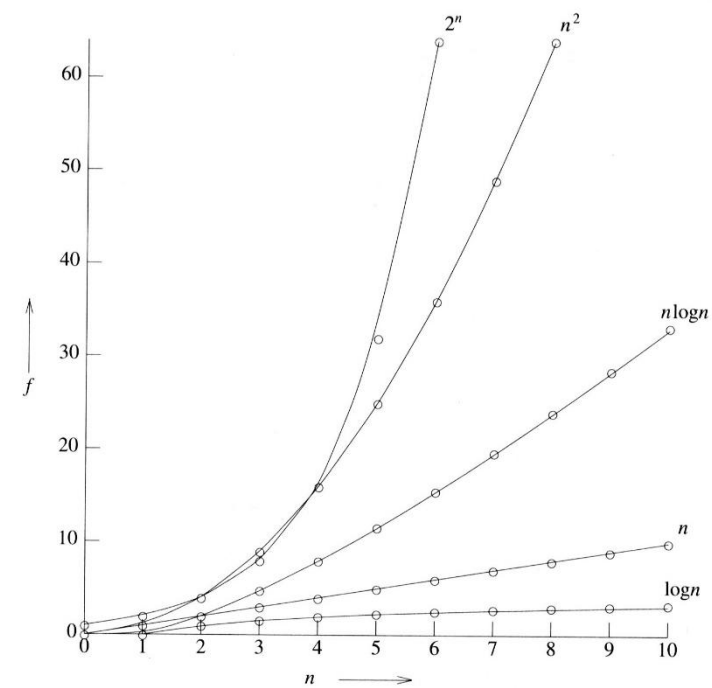
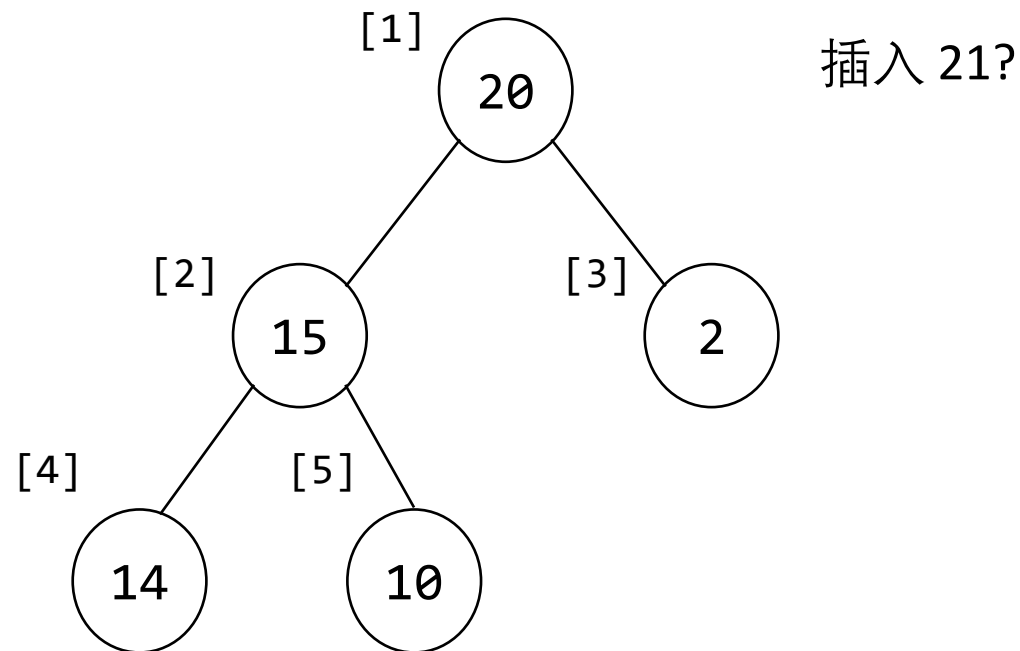


Figure 1.8 Plot of function values

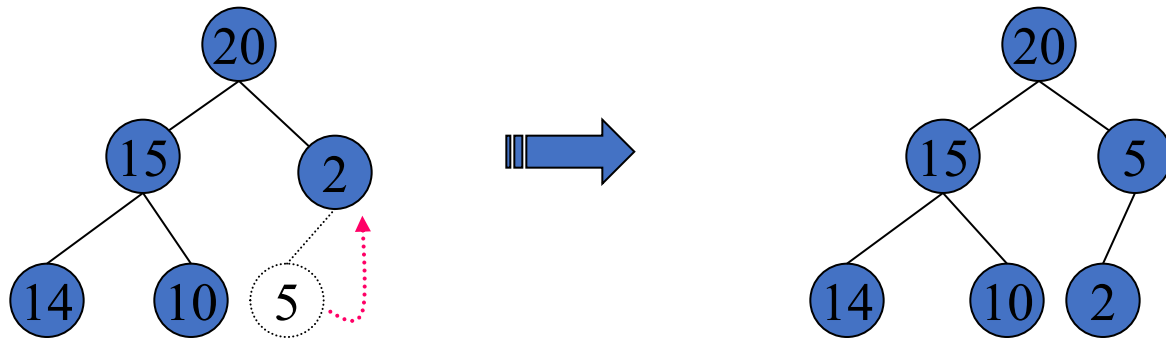
# 最大堆的实现

```
#define MAX_ELEMENTS 200 /* maximum size of heap+1 */
#define HEAP_FULL(n) (n == MAX_ELEMENTS-1)
#define HEAP_EMPTY(n) (!n)
typedef struct {
    int key;
    /* other fields */
} element;
element heap[MAX_ELEMENTS];
int n = 0;
```

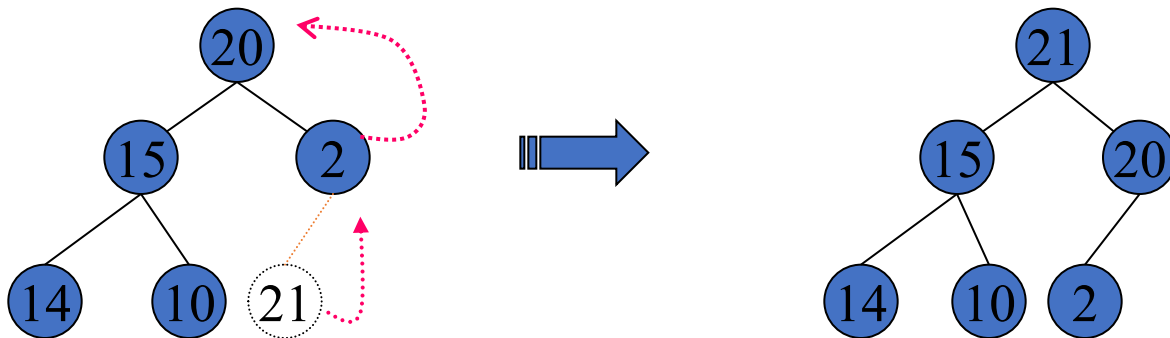
| [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] |
|-----|-----|-----|-----|-----|-----|-----|-----|
|     | 20  | 15  | 2   | 14  | 10  |     |     |



# 最大堆的插入操作



上浮



# 向最大堆中插入元素

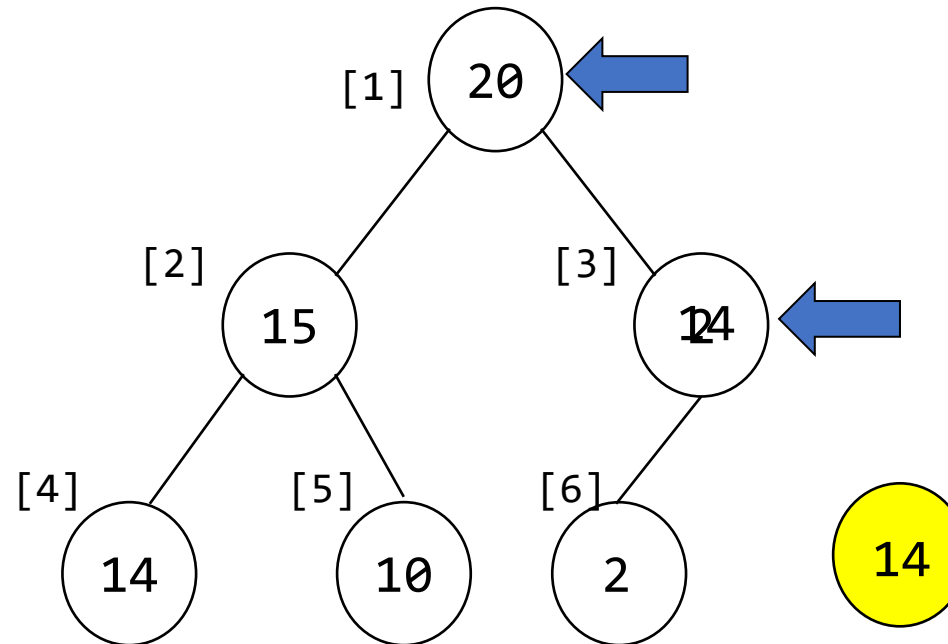
```
#define MAX_ELEMENTS 200    /* maximum heap size + 1 */
#define HEAP_FULL(n) (n==MAX_ELEMENTS-1)
#define HEAP_EMPTY(n) (!n)
typedef struct{
    int key;
    /* other fields */
}element;
element heap[MAX_ELEMENTS];
int n=0;

void push(element item, int *n)
{
    /* insert item into a max heap of current size * n */
    int i;
    if (HEAP_FULL(*n)) {
        fprintf(stderr, "the heap is full.\n");
        exit( EXIT_FAILURE );
    }
    i = ++(*n);
    while ( (i!=1) && (item.key>heap[i/2].key) ) {
        heap[i] = heap[i/2];
        i /= 2;
    }
    heap[i]= item;
}
```

## 示例

```
element e;  
e.key=14;  
no=5;  
push(e, &no);
```

```
void push(element item, int *n)  
{  
    int i;  
    if (HEAP_FULL(*n)) {  
        fprintf(stderr, "The heap is full. ");  
        exit(EXIT_FAILURE);  
    }  
    i = ++(*n);  
    while ((i != 1) && (item.key > heap[i/2].key)) {  
        heap[i] = heap[i/2];  
        i /= 2;  
    }  
    heap[i] = item;  
}
```



# push操作(性能)分析

❖ 构造一个 $n$ 个节点的二叉树

⇒ 二叉树高度为  $\lceil \log_2(n+1) \rceil$

一个有  $n$  个节点的完全二叉树的高度为  $\log_2(n+1)$ ,  
其中  $\lceil x \rceil$  表示大于等于  $x$  的最小整数。

由前述二叉树性质,可知:

$$n = 2^k - 1$$

$$n + 1 = 2^k$$

$$\log_2(n + 1) = k$$

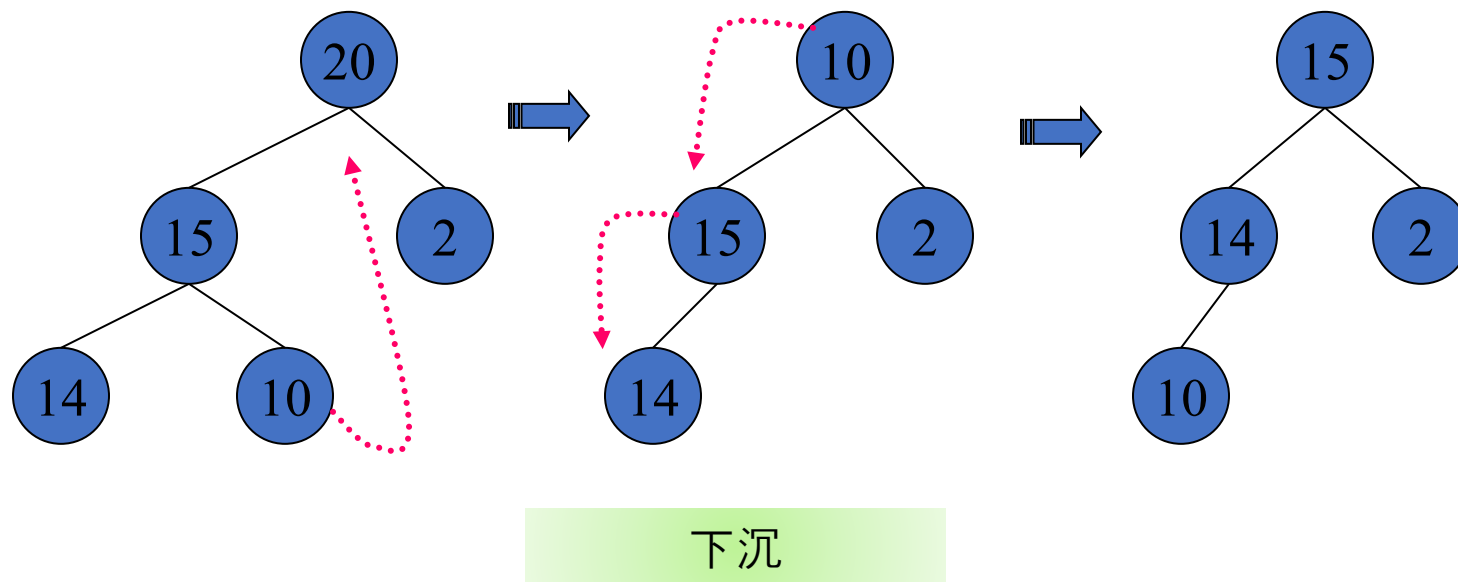
❖ while循环迭代  $(\log_2 n)$  次.

⇒ push操作的复杂度为  $O(\log_2 n)$



# 最大堆删除元素

- ❖ 当要从最大堆中删除一个元素时，会从堆的根部将其删除
- ❖ 删除元素 '20'

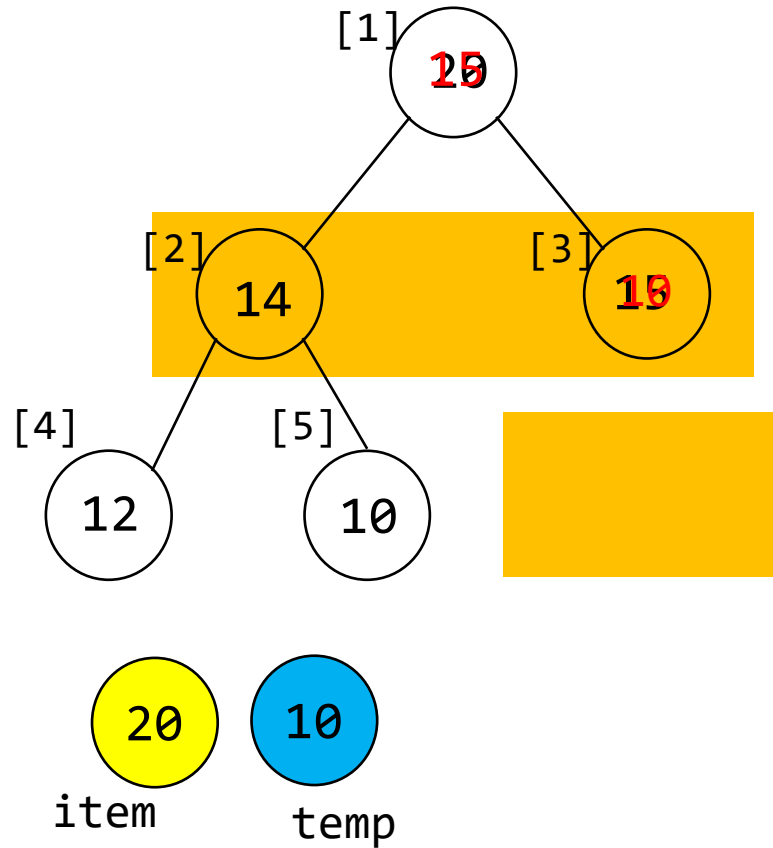


# 最大堆删除元素操作

```
element pop(int *n)
{
    /*从堆中删除键值最大的元素*/
    int parent, child;
    element item, temp;
    if (HEAP_EMPTY(*n))
    {
        fprintf(stderr, "堆空.\n");
        exit(EXIT_FAILURE);
    }
    item = heap[1];          /*保存键值最大的元素的值*/
    temp = heap[(*n)--];     /*使用堆中的最后一个元素调整堆*/
    parent = 1;
    child = 2;
    while (child <= *n)
    {
        /*找到当前父节点的较大子节点*/
        if ( (child < *n) && (heap[child].key < heap[child+1].key) )
            child++;
        if ( temp.key >= heap[child].key )
            break;
        heap[parent] = heap[child]; /*移动到下一较低层级*/
        parent = child;
        child *= 2;
    }
    heap[parent] = temp;
    return item;
}
```

## 示例

```
int no=5;  
element e=pop(&no);
```

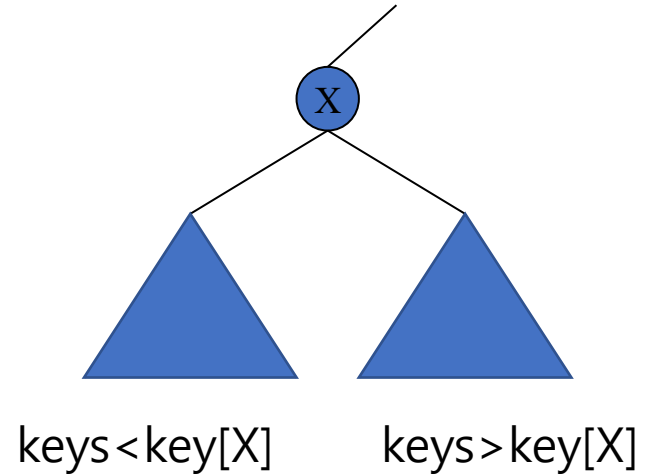


```
element pop(int *n)  
{  
    int parent, child;  
    element item, temp;  
    if (HEAP_EMPTY(*n)) {  
        fprintf(stderr, "The heap is empty");  
        exit(EXIT_FAILURE);  
    }  
    item = heap[1];  
    temp = heap[*n--];  
    parent = 1;  
    child = 2;  
    while (child <= *n) {  
        if((child < *n) &&  
            (heap[child].key < heap[child+1].key))  
            child++;  
        if (temp.key >= heap[child].key) break;  
        heap[parent] = heap[child];  
        parent = child;  
        child *= 2;  
    }  
    heap[parent] = temp;  
    return item;  
}
```

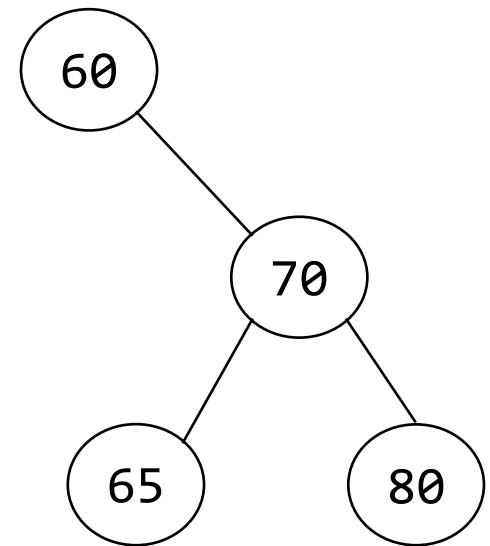
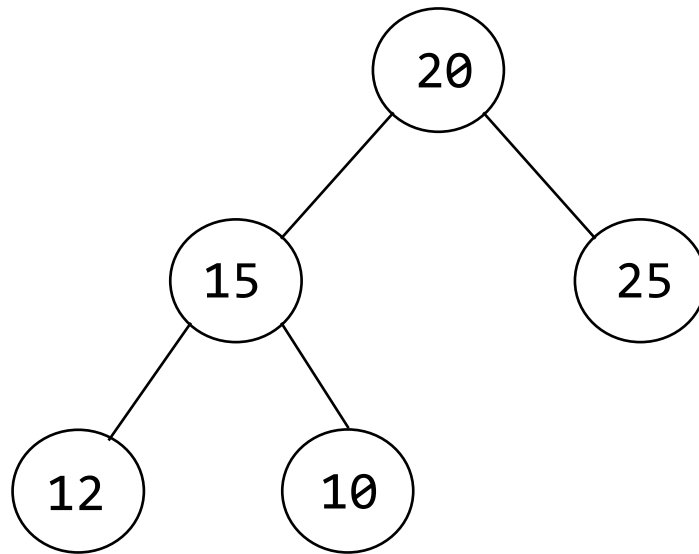
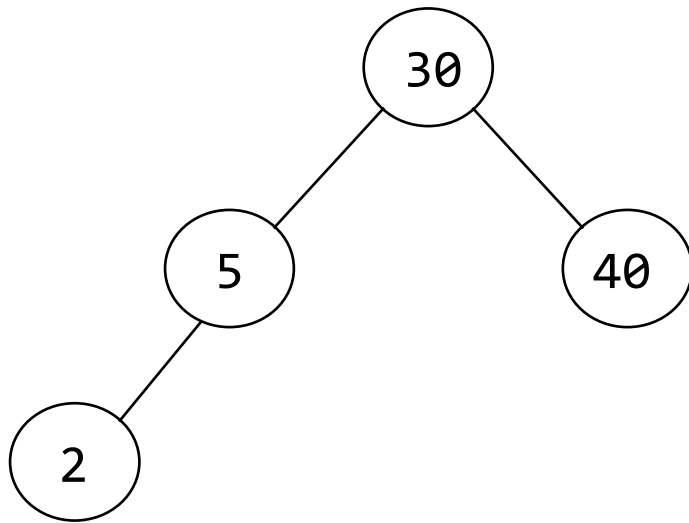
# 二叉搜索树(BST)

❖ 二叉搜索树是二叉树。它可能为空。如果不为空，那么它满足如下属性：

- 1) 每个节点恰好有一个键，且树中的键各不相同
- 2) 左子树中的键（如果有）小于根节点的键
- 3) 右子树中的键（如果有）大于根节点的键
- 4) 左右子树也是二叉搜索树



# 二叉搜索树示例

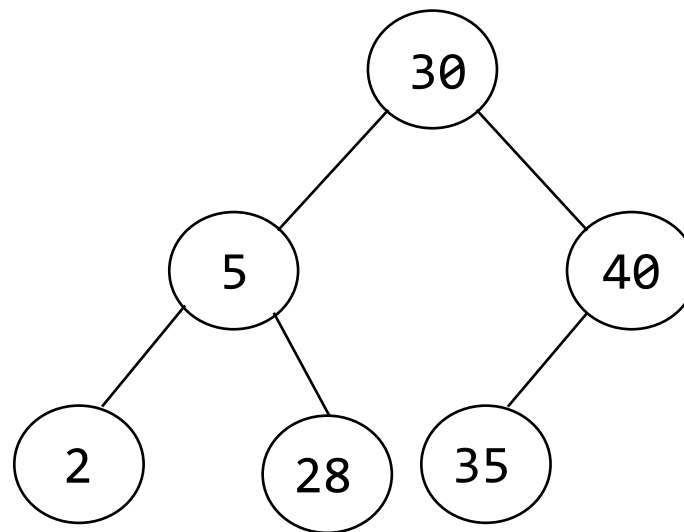


# 二叉搜索树特点

- 搜索, 插入和删除操作受限于  $O(h)$ , 其中  $h$  是二叉搜索树的高度
- 这些操作可以通过以下两种方式执行:
  - 按键值  
e.g.) 删除键值为x的元素
  - 按排名  
e.g.) 删除第5小的元素
- 二叉搜索树的中序遍历生成一个有序列表

中序遍历

2   5   28   30   35   40



# 搜索二叉搜索树

```
element* search(treePointer root, int k)
{
    /*返回键值为 k 的元素指针, 如果没有, 则返回 NULL */

    if ( !root ) return NULL;
    if ( k == root->data.key ) return &(amp;root->data);
    if ( k < root->data.key )
        return search(root->leftChild, k);
    return search( root->rightChild, k );
}
```

递归搜索

```
typedef struct element {
    int key;
};

typedef struct node *treePointer;
typedef struct node {
    element data;
    treePointer leftChild;
    treePointer rightChild;
};
```

```
element* iterSearch(treePointer tree, int k)
{
    /*返回键值为 k 的元素指针、 如果没有这样的元素, 则返回 NULL */
    while( tree ) {
        if ( k == tree->data.key ) return &(amp;tree->data);
        if ( k < tree->data.key )
            tree = tree->leftChild;
        else
            tree = tree->rightChild;
    }
    return NULL;
}
```

迭代搜索

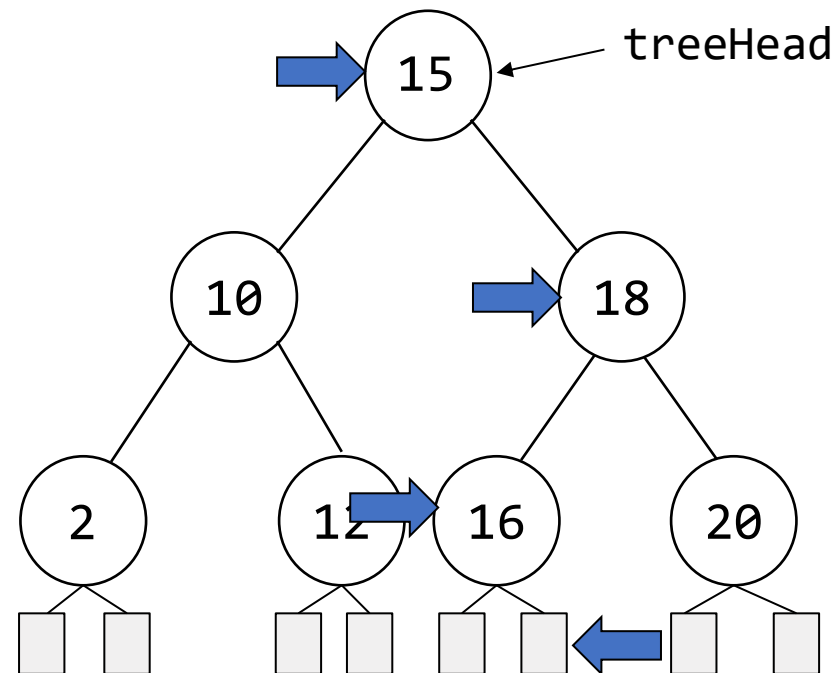
## 示例

```
treePointer tr = search(treeHead, 17);
```

树中无17元素

|                                                                                                 |      |
|-------------------------------------------------------------------------------------------------|------|
| search(  , 17) | NULL |
| search( 16 , 17)                                                                                | NULL |
| search( 18 , 17)                                                                                | NULL |
| search( 15 , 17)                                                                                | NULL |
| main()                                                                                          | NULL |

Function call stack



```
element* search(treePointer root, int k)
{
    /* return a pointer to the element whose key is k,
       if there is no such element, return NULL */

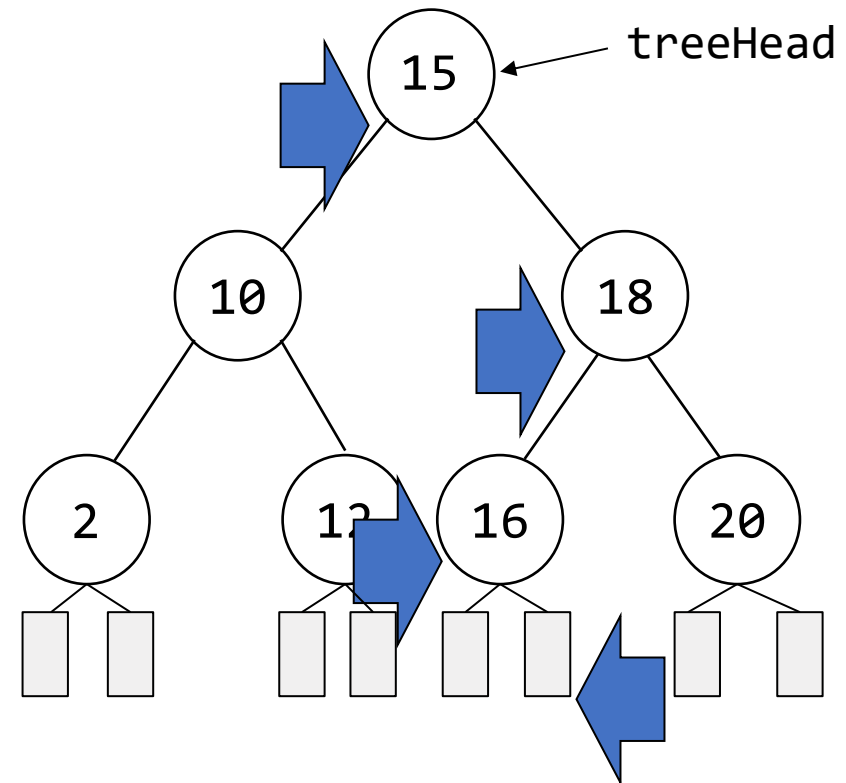
    if ( !root ) return NULL;
    if ( k == root->data.key ) return &(root->data);
    if ( k < root->data.key )
        return search(root->leftChild, k);
    return search( root->rightChild, k );
}
```



## 示例

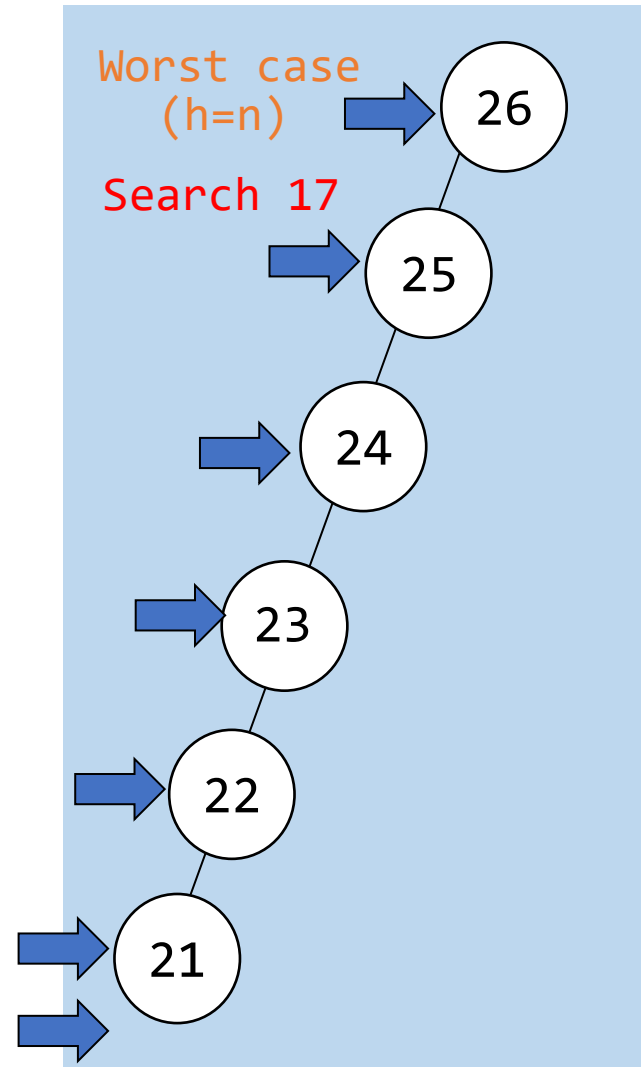
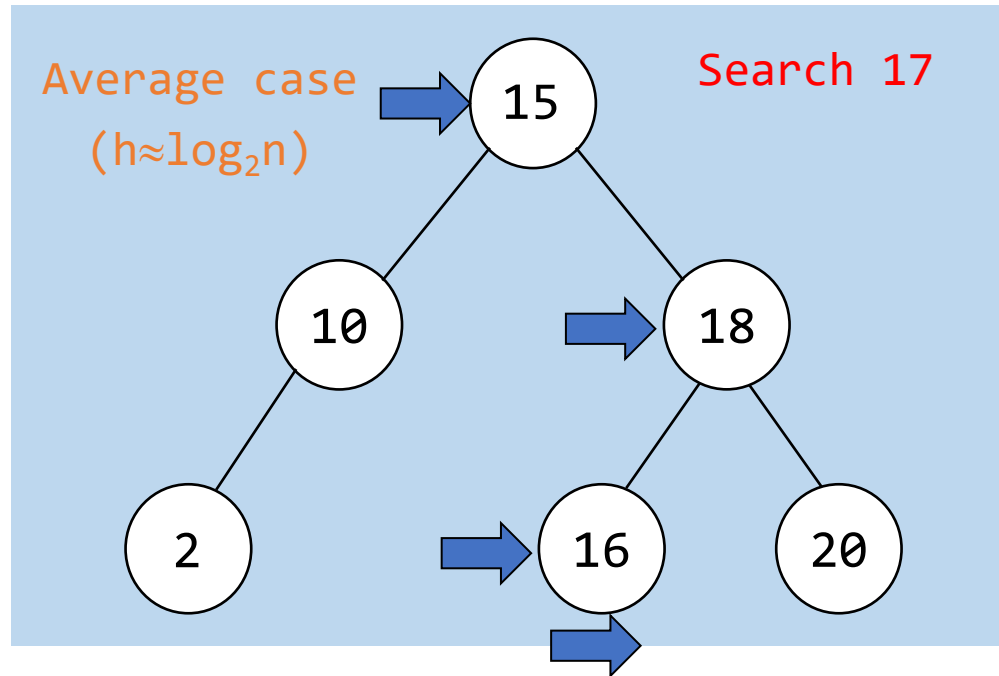
```
treePointer tr = iterSearch(treeHead, 17);
```

```
element* iterSearch (treePointer tree, int k)
{
    while (tree) {
        if (k == tree->data.key) return &(tree->data);
        if (k < tree->data.key)
            tree = tree->leftChild;
        else
            tree = tree->rightChild;
    } /* while */
    return NULL;
}
```



# 二分搜索树的搜索操作时间复杂度

- 平均情况 (Average case)  
 $O(h)$ ,  $h$  是BST的高度
- 最坏情况 (Worst case)  
 $O(n)$ ,  $n$  是节点个数

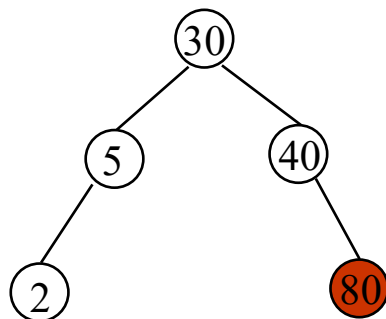
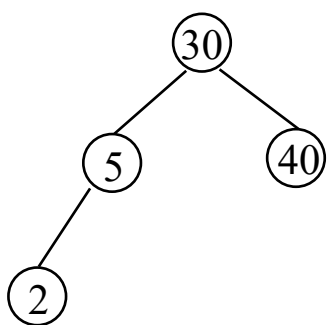


# 向二叉搜索树中插入

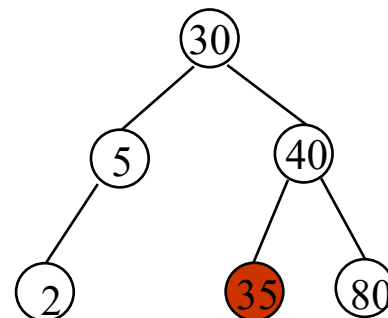
## ❖ 向树中插入键值80

- 首先在树种查找80
- 最后检查的节点有一个键值40
- 插入关键字80作为节点的右子节点

## ❖ 在生成的树中插入键值35



Insert 80



Insert 35

# 向二叉搜索树中插入

```
void insert (treePointer *node, int k)
{
    /* if k is in the tree pointed at by node do nothing;
       otherwise add a new node with data =(k) */
    treePointer ptr;
    /* searches the binary search tree *node for the key k */
    treePointer temp = modifiedSearch( *node, k );
    if ( temp || !(*node) ) {
        /* k is not in the tree */
        MALLOC( ptr, sizeof(*ptr) );
        ptr->data.key = k;
        ptr->leftChild = ptr->rightChild = NULL;

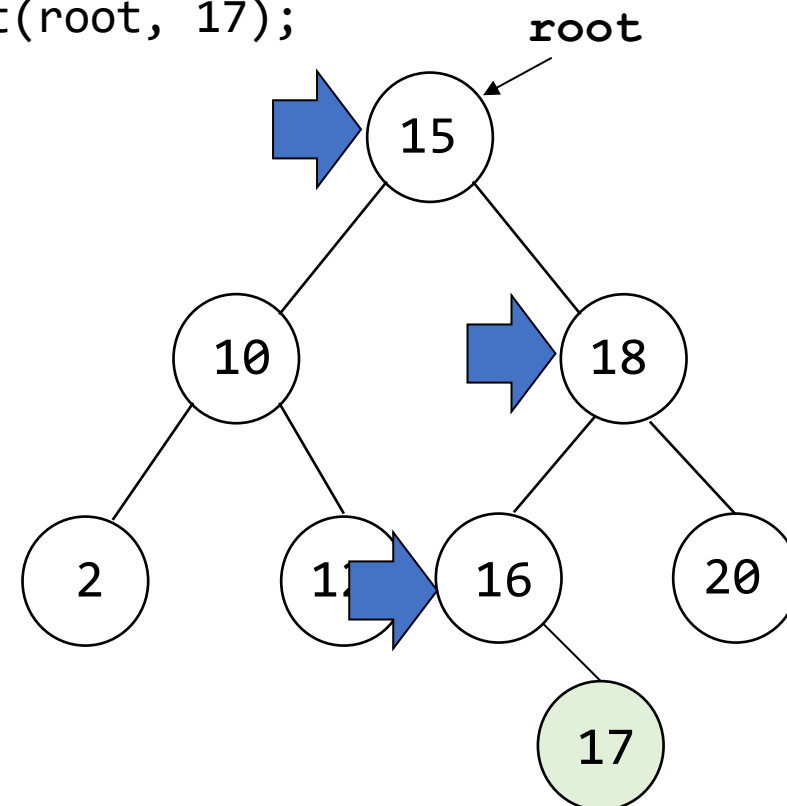
        if ( *node ) /* insert as child of temp */
            if (k<temp->data.key)
                temp->leftChild = ptr;
            else
                temp->rightChild = ptr;
        else
            *node = ptr;
    }
}
```

```
modifiedSearch():
if (a tree is empty || k is in the
tree)
    return NULL
else
    return 搜索过程中最后检查的节点的指针
```

# 二叉搜索树中插入

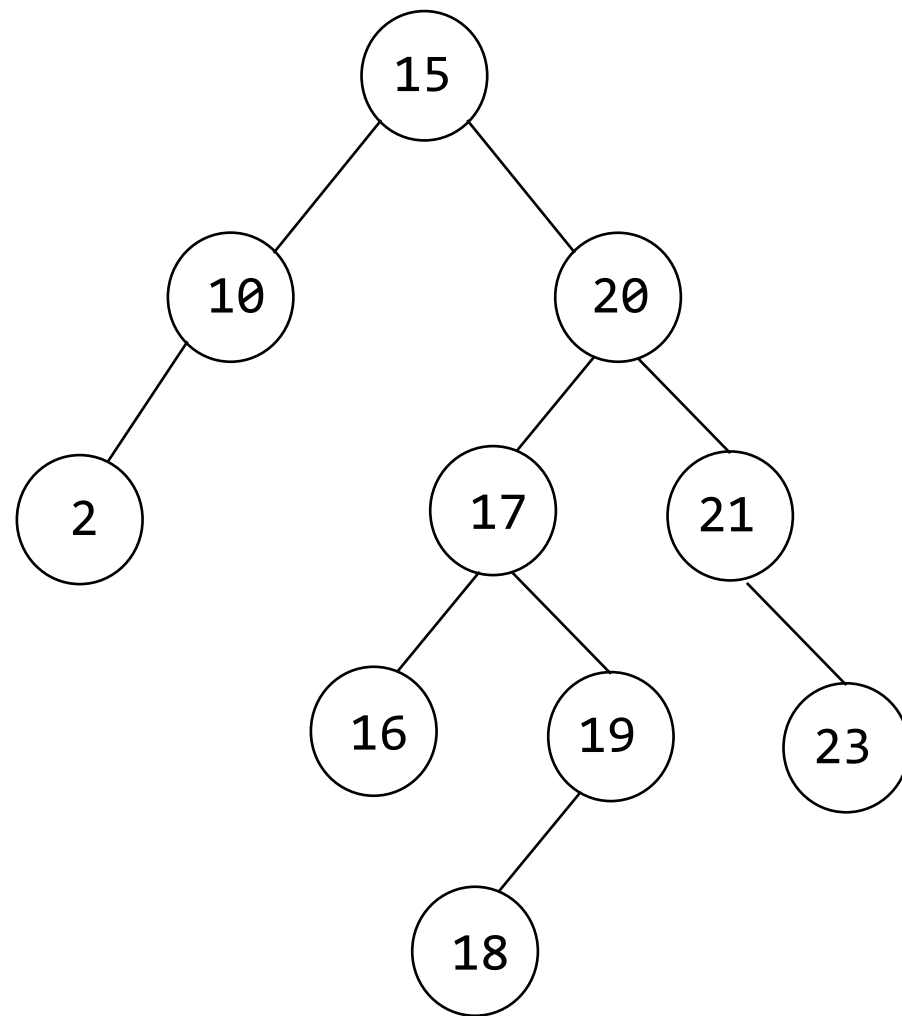
```
void insert(treePointer *node, int k) {  
    treePointer ptr, temp = modifiedSearch(*node, k);  
    if (temp || !(*node)) {  
        MALLOC(ptr, sizeof (*ptr));  
        ptr->data.key = k;  
        ptr->leftChild = ptr->rightChild = NULL;  
        if (*node) {  
            if (k < temp->data.key) {  
                temp->leftChild = ptr;  
            } else {  
                temp->rightChild = ptr; }  
        } else { *node = ptr; }  
    }  
}
```

insert(root, 17);

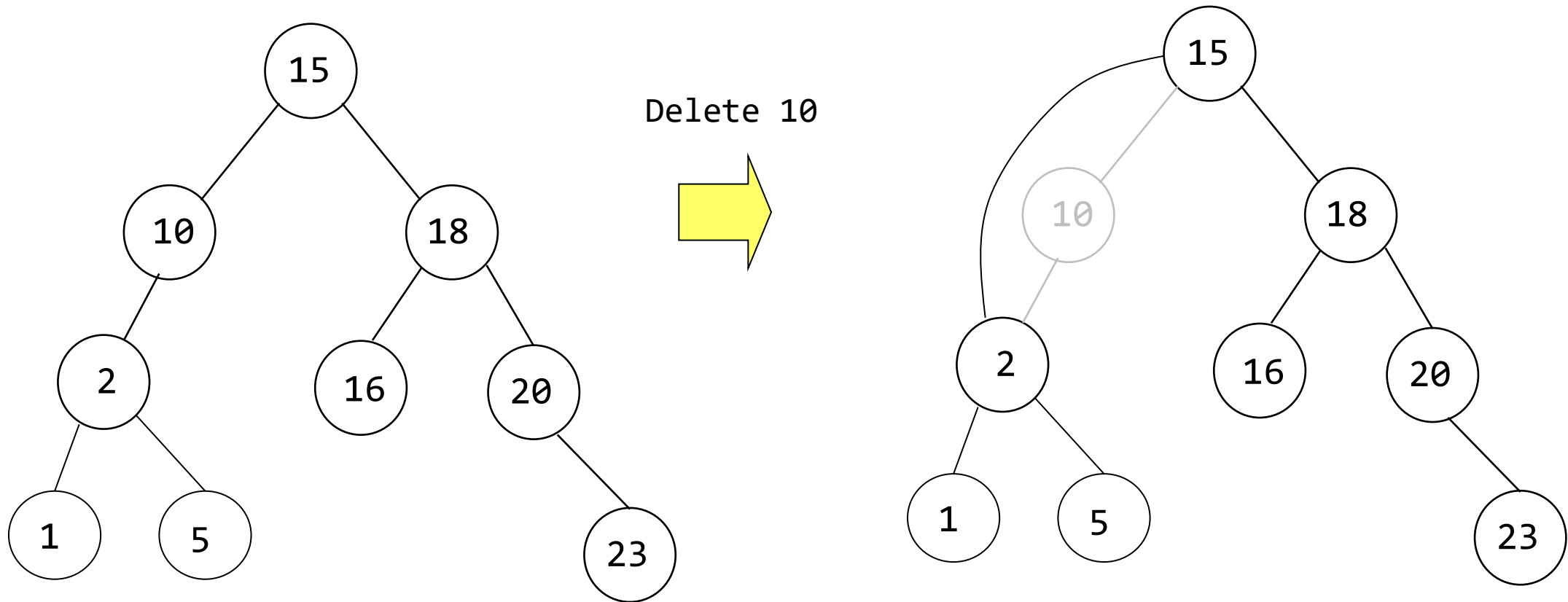


# 二叉搜索树中删除

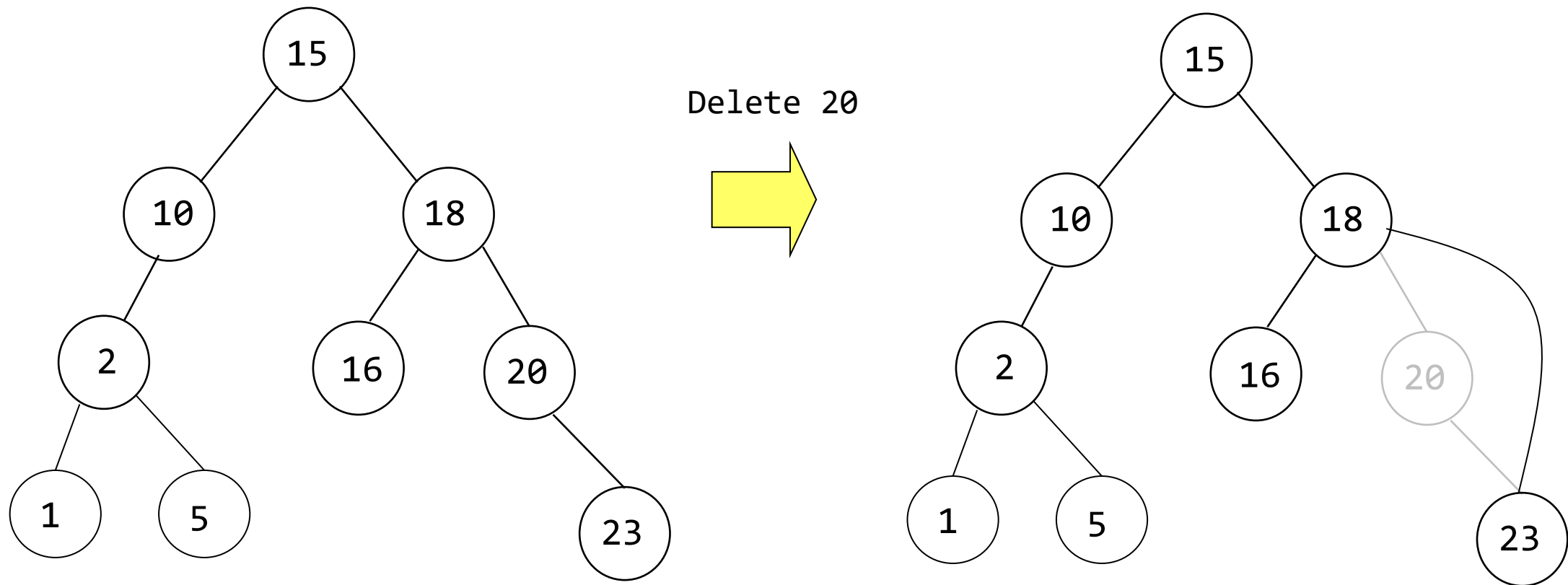
- 删除一个叶子节点
- 删除有1个子节点的节点
- 删除有2个子节点的节点



# 删除有一个子节点的节点

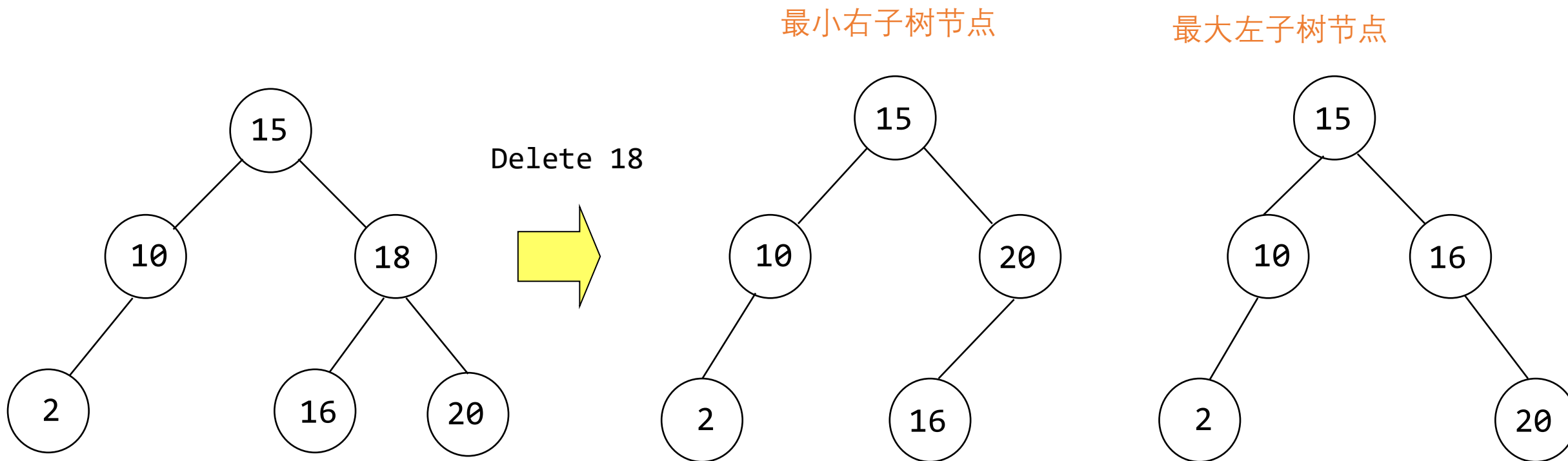


# 删除有一个子节点的节点





# 删除有2个子节点的节点



→ 时间复杂度： $O(h)$ ，其中  $h$  是二叉搜索树的高度

# 二叉搜索树的高度

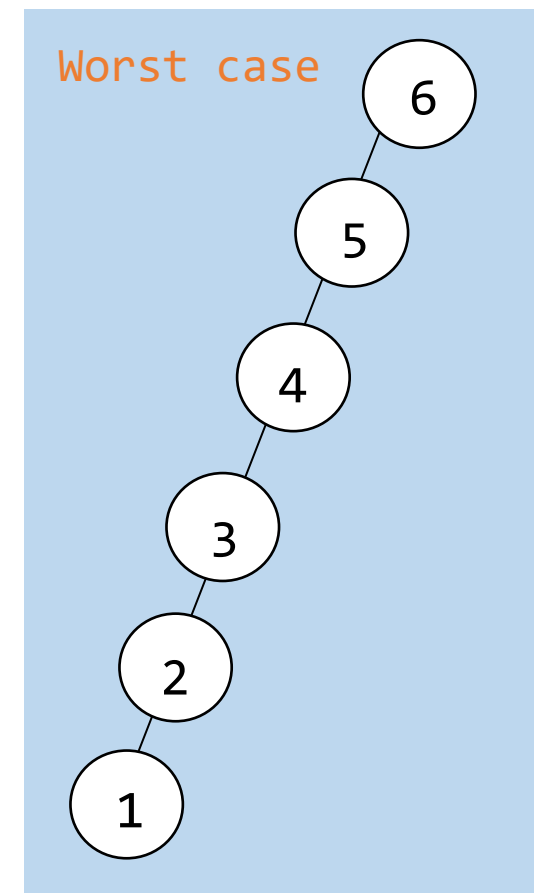
## ❖ 具有 $n$ 个元素的BST的高度

- 平均情况(average case):  $O(\log_2 n)$
- 最坏情况(worst case):  $O(n)$

e.g.) 使用 `insert()` 在初始为空的BST中插入键 $1, 2, 3, \dots, n$

## ❖ 平衡搜索树(BST: Balanced Search Trees)

- 最坏情况高度 :  $O(\log_2 n)$
  - 搜索, 插入和删除操作受限于  $O(h)$ , 其中  $h$  是二叉树的高度
- e.g.) AVL tree(平衡二叉树), 2-3 tree, red-black tree(红黑树)



# 下节课内容

图

